Aprende Git Interactivo: Plataforma de Simulación Visual y Tutoría Inteligente para la Enseñanza y Comprensión del Control de Versiones

Jonathan Torres, Ricardo Riera y Javier Darder (Grupo 6)

Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela

Diseño y Prototipado de Recursos Educativos Digitales (DPRED): Informe del Proyecto, Fases I y II

Prof. Felix Urbano y Profa. Yusneyi Carballo Barrera

19 de octubre de 2026

Contenido

PARTE I — FASE I: ANÁLISIS Y DISEÑO (corregida según la retroalimentación docente)

1. Definición del Problema y Delimitación del Alcance
2. Metodología para la Elaboración del Recurso Educativo Digital
3. Análisis del Entorno (PESTEL)
4. Análisis Estratégico Interno y Externo (FODA)
5. Análisis de Usuarios, Usuarias y Clientela (User Personas)
6. Análisis de la Competencia (Benchmarking)
7. Propuesta de Solución, Justificación Pedagógica y Arquitectura
8. Encaje de Valor (Value Proposition Canvas)

PARTE II — FASE II: DISEÑO INSTRUCCIONAL, DISEÑO INFORMÁTICO Y DESARROLLO

9. Diseño Instruccional
10. Diseño Informático
11. Validación y Correspondencia con el Prototipo Funcional
12. Declaración de Uso de Inteligencia Artificial Generativa (IAG)

Referencias

PARTE I — FASE I: ANÁLISIS Y DISEÑO

1. Definición del Problema y Delimitación del Alcance

En la formación universitaria de ingenieros y licenciados en computación, el dominio de sistemas de control de versiones distribuido —con Git como estándar de facto de la industria— constituye una competencia profesional transversal básica no negociable (Spinellis, 2012). No obstante, investigaciones pedagógicas en enseñanza de la computación (Haaranen y Lehtinen, 2015; Zagalsky et al., 2015) demuestran de manera consistente que los estudiantes experimentan severas dificultades para asimilar el modelo conceptual de Git, recurriendo frecuentemente a la memorización mecánica de sintaxis desarticulada.

A partir de observaciones diagnósticas de campo y la experiencia docente acumulada en la Escuela de Computación de la Universidad Central de Venezuela (UCV), se plantea como **hipótesis diagnóstica de partida** que una proporción sustancial de estudiantes en sus primeras asignaturas de desarrollo grupal (tales como Algoritmos y Programación, Programación Orientada a Objetos e Ingeniería de Software) no logra consolidar el modelo mental de Git. El alumnado suele memorizar comandos sintácticos (*git add, git commit, git push*) sin comprender la arquitectura de estados internos (Working Directory, Staging Area, Local Repository) ni la estructura subyacente de Grafo Acíclico Dirigido (DAG) que fundamenta el historial de cambios (Chacon y Straub, 2014).

Esta brecha formativa desemboca en la denominada «ansiedad ante la terminal» (*CLI anxiety*), parálisis frente a mensajes de error en inglés y colapso de repositorios compartidos en asignaturas de desarrollo grupal. Frente a este panorama, el problema no reside en la falta de documentación, sino en la ausencia de herramientas didácticas interactivas y contextualizadas en español que hagan visible el modelo mental abstracto de Git.

Delimitación Poblacional e Institucional: La población de estudio y beneficiaria directa está conformada por estudiantes de pregrado de la Escuela de Computación (Facultad de Ciencias, UCV) matriculados en materias que demandan integración continua y trabajo colaborativo de código, con una población secundaria compuesta por los docentes de dichas asignaturas, quienes requieren herramientas confiables de andamiaje para mitigar fallas recurrentes en los proyectos grupales.

1.1. Delimitación del Alcance y Definición del Producto Mínimo Viable (MVP)

Para favorecer la viabilidad pedagógica, técnica y temporal en el marco del período semestral de la cátedra DPRED, el proyecto adopta los principios de formulación de Producto Mínimo Viable en contextos académicos de innovación (Ospina y Carballo Barrera, 2022), estructurando el alcance en un Roadmap de tres fases bien diferenciadas:

- **Fase 1 — Producto Mínimo Viable (MVP Semestral DPRED):** Comprende la totalidad del Módulo 1 (El Core: simulación de áreas de trabajo locales y Grafo Vivo interactivo en tiempo real con renderizado de nodos de commit, ramas y puntero HEAD), junto con una experiencia esencial y guiada de resolución de conflictos locales entre dos ramas (subconjunto prioritario del Módulo 3). Incluye terminal simulada en el navegador, asistencia mediante reglas de andamiaje socrático determinista y metas de accesibilidad digital. Durante el desarrollo (Fase II) se incorporaron además seis lecciones animadas y una versión simulada del Módulo 2 (push, Pull Requests y revisión sin conexión real con GitHub), sin servicios externos obligatorios.
- **Fase 2 — Extensiones Evolutivas de Colaboración (Post-MVP):** Registro de usuarios con progreso almacenado en un servidor, integración opcional con la API pública de GitHub

mediante autenticación OAuth para portafolios y ampliación del Módulo 2 con escenarios de colaboración adicionales.

- **Fase 3 — Tutoría Cognitiva Avanzada y Operaciones Complejas (Evolución Futura):** Módulo 3 completo (simulador avanzado de Merge vs. Rebase interactivo, diagnóstico de historiales rotos mediante simulación de reflog) y tutoría dinámica con modelos generativos de lenguaje externos con canal socrático en tiempo real.
- **Exclusiones Explícitas del Proyecto:** El RED no contempla emular la especificación completa de Git (como comandos internos de bajo nivel tipo plumbing: cat-file, hash-object, ni hooks de servidor o submódulos complejos), sino consolidar con alta precisión pedagógica los flujos de trabajo profesionales fundamentales (Feature Branching y Trunk-based) requeridos en el ámbito universitario y laboral junior.

1.2. Objetivos del Recurso Educativo Digital (RED)

Con la finalidad de guiar el proceso de ingeniería y evaluación con metas medibles y verificables, se establecen los siguientes objetivos:

Objetivo General: Diseñar y validar un prototipo web interactivo (Recurso Educativo Digital) que facilite la comprensión conceptual y el modelado mental de los flujos fundamentales de control de versiones con Git en estudiantes de la Licenciatura en Computación de la Universidad Central de Venezuela, mediante simulación gráfica visual y retroalimentación formativa inmediata.

Objetivos Específicos:

- **1.** Diagnosticar las principales dificultades cognitivas, barreras sintácticas y concepciones erróneas que presentan los estudiantes de la Escuela de Computación (UCV) al interactuar con el sistema de control de versiones Git.

- **2.** Fundamentar pedagógicamente el recurso bajo los principios del constructivismo interactivo, el aprendizaje significativo y el Diseño Universal para el Aprendizaje (DUA), procurando su pertinencia didáctica.
- **3.** Diseñar la arquitectura de interacción, la interfaz gráfica de usuario y las directrices de accesibilidad digital orientadas al cumplimiento de las pautas WCAG 2.2 nivel AA.
- **4.** Desarrollar un Producto Mínimo Viable (MVP) ejecutable en el navegador que integre la emulación del motor local de Git, el renderizado reactivo del grafo de commits y un simulador didáctico de conflictos locales.
- **5.** Evaluar la usabilidad, carga cognitiva percibida y efectividad formativa del prototipo con estudiantes y docentes de asignaturas de desarrollo de software en la UCV.

2. Metodología para la Elaboración del Recurso Educativo Digital

Para responder a las exigencias metodológicas del diseño instruccional y la ingeniería de software educativo (Gros Salvat, 1997; Herrera Batista, 2006), el proyecto adopta un enfoque metodológico híbrido que articula el modelo instruccional **ADDIE** (Análisis, Diseño, Desarrollo, Implementación y Evaluación) con el marco de **Diseño Centrado en el Usuario (DCU / Design Thinking)** (Brown, 2008; Costa Montenegro et al., 2016). Este modelo asegura rigor pedagógico, empatía con las necesidades reales del estudiante y entregables estructurados a lo largo de las fases de la asignatura.

Tabla 1

Metodología de elaboración del RED articulando ADDIE y DCU.

Etapa ADDIE / DCU	Objetivo y Actividades Clave	Entregables del Proyecto	Evidencias y Validación
1. Análisis (Empatizar y Definir)	Investigación de necesidades formativas; benchmarking de alternativas existentes; análisis PESTEL y FODA; modelado de User Persona y Journey Map.	Informe de Análisis Preliminar (Entrega #1); Arquetipos de usuario; Matriz de encaje de valor.	Observaciones de aula UCV; encuestas diagnósticas preliminares; matriz de benchmarking comparativo.
2. Diseño (Idear y Conceptualizar)	Definición de objetivos de aprendizaje; diseño instruccional de niveles y andamiaje; arquitectura de información; wireframes y mockups con criterios DUA/WCAG.	Informe de Diseño Detallado (Entrega #2); Mockup navegable en herramienta de prototipado; Guía de estilo accesible.	Especificación de interacción; árbol de navegación; rúbricas de contraste y accesibilidad.
3. Desarrollo (Prototipar)	Construcción del Producto Mínimo Viable (MVP): motor de estados de Git en JavaScript, renderizador de grafo dinámico en Canvas/SVG y reglas heurísticas de tutoría Nova.	Código fuente del prototipo funcional; despliegue web del MVP en entorno de pruebas (Netlify).	Repositorio Git con control de versiones; bitácora de commits; pruebas unitarias del simulador de comandos.
4. Implementación (Prueba Piloto)	Despliegue controlado del prototipo para pruebas con la muestra seleccionada de estudiantes y docentes de la Escuela de Computación.	Guía de usuario y manual pedagógico para el docente; entorno de pruebas en línea operativo.	Registros de interacción y sesiones piloto de uso en laboratorios de la universidad.
5. Evaluación (Testear y Ajustar)	Evaluación de usabilidad (SUS - System Usability Scale); medición de carga cognitiva; evaluación de accesibilidad técnica (Lighthouse/WAVE) y feedback docente.	Informe Final de Evaluación y Validación (Entrega #3); plan de mejoras y roadmap post-semestre.	Resultados estadísticos de usabilidad; matriz de hallazgos y correcciones aplicadas.

3. Análisis del Entorno (PESTEL)

Para asegurar la viabilidad contextual y sostenibilidad de la plataforma, se realiza un análisis de las seis dimensiones macroambientales, sustentando las condiciones políticas y normativas en marcos jurídicos verificables y matizando supuestos tecnológicos y ambientales:

- **Dimensión Política (P):** Promoción institucional y regional de competencias digitales y educación STEM. En el ámbito venezolano, la Ley Orgánica de Ciencia, Tecnología e Innovación (LOCTI) y la Ley de Infogobierno (República Bolivariana de Venezuela, 2013)

establecen la primacía del software libre y los estándares abiertos en las instituciones públicas y de educación superior. A nivel internacional, se alinea con la Recomendación sobre los Recursos Educativos Abiertos (REA) de la UNESCO (2019). Riesgo político bajo de restricciones sobre servicios de distribución de código abierto.

- **Dimensión Económica (E):** Restricciones presupuestarias severas en la educación universitaria pública venezolana y altos costos de licencias de software comercial. Frente a esto, el modelo arquitectónico se diseña para minimizar costos de infraestructura: al ejecutar el motor de simulación de Git directamente en el navegador del cliente mediante JavaScript/WebAssembly, se reducen drásticamente los requerimientos de computación centralizada en backend. El alojamiento estático se sostiene bajo capas gratuitas institucionales (GitHub Pages, Vercel Hobby tier), manteniendo los costos operativos recurrentes en niveles marginales cercanos a cero para la universidad (IBM, s.f.).
- **Dimensión Social (S):** Las nuevas cohortes de estudiantes demandan experiencias activas de aprendizaje interactivo y gamificado («learn-by-doing»), mostrando rechazo hacia manuales estáticos o videos teóricos pasivos. La interfaz busca mitigar la «ansiedad ante la terminal» (CLI anxiety) mediante representaciones visuales humanizadas, diseño empático y soporte 100% en idioma español, superando la barrera idiomática que presentan la mayoría de recursos técnicos oficiales.
- **Dimensión Tecnológica (T):** La madurez de los estándares web modernos (HTML5 Canvas, SVG reactivo, Web Workers y WebAssembly) permite virtualizar un sistema de archivos y un motor simplificado de control de versiones en el cliente sin requerir instalación local ni privilegios de administrador en salas de laboratorio universitarias. La arquitectura se

desacopla de APIs externas en su MVP para evitar caídas de servicio ante fluctuaciones de conectividad (Chacon y Straub, 2014).

- **Dimensión Ecológica / Ambiental (E):** Adopción de principios de Green Computing y computación sostenible. Al procesar los estados de Git del lado del cliente y optimizar las cargas de red (JAMstack), se reduce la demanda de procesamiento constante en servidores y centros de datos. Asimismo, elimina la dependencia de guías y manuales impresos de laboratorio. Si bien no elimina de manera absoluta el consumo energético intrínseco del dispositivo del usuario ni la infraestructura de red global, disminuye sensiblemente la huella de carbono asociada al cómputo dedicado.
- **Dimensión Legal (L):** Cumplimiento del marco normativo venezolano, específicamente la Constitución de la República Bolivariana de Venezuela (Art. 28 sobre protección del honor y datos personales) y la Ley sobre Mensajes de Datos y Firmas Electrónicas (República Bolivariana de Venezuela, 2001). A nivel de mejores prácticas internacionales, se contemplan los principios de minimización de datos del Reglamento General de Protección de Datos (RGPD / GDPR de la Unión Europea). El MVP operará sin recolectar datos personales ni credenciales. Para fases posteriores que integren la API de GitHub, se acatarán estrictamente los Términos de Servicio de la API de GitHub Developer, incorporando un descargo de responsabilidad (disclaimer) legal explícito que aclare que la plataforma es un proyecto académico independiente sin filiación oficial con Microsoft o GitHub.

4. Análisis Estratégico Interno y Externo (FODA)

Para articular las capacidades internas del equipo con los desafíos del entorno, se presenta la matriz FODA y, de forma complementaria, la matriz de estrategias cruzadas resultantes (FO, DO, FA, DA):

Tabla 2

Matriz FODA del proyecto Aprende Git Interactivo.

ORIGEN INTERNO (Controlable por el equipo)	ORIGEN EXTERNO (Entorno macro y micro)
FORTALEZAS (F) 1. Interactividad Real: Simulación visual y gráfica inmediata superior a manuales y videos pasivos. 2. Arquitectura Client-Side: Ejecución en el navegador con costos de infraestructura marginales y alta velocidad. 3. Andamiaje Pedagógico Integrado: Asistencia heurística contextual mediante la tutora Nova para guiar en atascos. 4. Enfoque DUA y Accesibilidad: Concepción nativa orientada a pautas WCAG 2.2 AA e inclusión en español.	OPORTUNIDADES (O) 1. Demanda Curricular Ineludible: Requerimiento mandatorio de Git en la carrera de Computación UCV y el mercado tech. 2. Adopción Académica Institucional: Posibilidad de convertirse en recurso didáctico recomendado en Ingeniería de Software. 3. Ecosistema de Software Libre: Facilidad para liberar el código del RED y recibir retroalimentación de la comunidad. 4. Infraestructura Gratuita para Educación: Uso de planes gratuitos de Netlify, Vercel o GitHub Pages y programas educativos.
DEBILIDADES (D) 1. Restricción Temporal Semestral: Cronograma académico limitado de una asignatura para abarcar la complejidad total de Git. 2. Presupuesto Cero: Carencia de fondos comerciales para infraestructura de servidores dedicados de cómputo intensivo. 3. Complejidad en la Emulación de Consola: Desafío técnico de compatibilizar una terminal accesible con lectores de pantalla. 4. Capacidad Operativa Acotada: Equipo conformado por tres estudiantes de pregrado con múltiples obligaciones académicas.	AMENAZAS (A) 1. Competidores Consolidados: Plataformas internacionales con alto tráfico web (Learngitbranching, Oh My Git!). 2. Frustración y Abandono Temprano: Riesgo de deserción si la interfaz o los niveles iniciales no guían adecuadamente. 3. Variabilidad en Políticas de APIs Externas: Dependencia de cambios en OAuth o políticas de GitHub si se integra tempranamente. 4. Sobreconfianza en Asistentes de IA Comerciales: Estudiantes que usan IA para generar comandos sin comprender su funcionamiento.

Tabla 3

Matriz de Estrategias Cruzadas derivadas del FODA.

Tipo de Estrategia	Acciones Estratégicas Formuladas
Estrategias FO (Fortalezas + Oportunidades)	• FO1: Promover el despliegue del prototipo client-side en las cátedras de programación de la UCV, aprovechando que no requiere presupuesto de servidores ni instalación en los laboratorios. • FO2: Difundir el recurso como software libre educativo bajo licencia permisiva, captando el interés de estudiantes de semestres superiores para enriquecer la biblioteca de ejercicios.
Estrategias DO (Debilidades + Oportunidades)	• DO1 (Priorización de MVP): Acotar estrictamente el alcance semestral al Módulo 1 y experiencia esencial de conflictos, asegurando la entrega de un producto robusto y validable en los tiempos académicos. • DO2: Emplear servicios de alojamiento estático gratuito con integración continua (publicación automática de Netlify desde GitHub), mitigando la ausencia de presupuesto de infraestructura.

Tipo de Estrategia	Acciones Estratégicas Formuladas
Estrategias FA (Fortalezas + Amenazas)	• FA1 (Diferenciación Didáctica): Posicionar a «Aprende Git Interactivo» frente a competidores en inglés mediante una interfaz íntegramente en español, diseño accesible DUA y orientación constructivista adaptada al estudiante venezolano. • FA2: Incorporar la tutoría Nova como un andamiaje socrático guiado que enseñe a reflexionar sobre el grafo, diferenciándose de las herramientas de IA generativa comercial que solo escupen comandos ciegos.
Estrategias DA (Debilidades + Amenazas)	• DA1 (Desacoplamiento Tecnológico): Diseñar el motor del MVP completamente independiente de APIs externas de terceros (OAuth/GitHub), eliminando el riesgo de quiebre funcional durante el semestre. • DA2: Diseñar una curva de dificultad progresiva y mecánicas de onboarding empático para reducir drásticamente la tasa de frustración y abandono inicial de los usuarios.

5. Análisis de Usuarios, Usuarias y Clientela (User Personas)

5.1. Metodología de Obtención y Validación del Perfil

Para sustentar la representatividad de los arquetipos formulados y evitar sesgos de diseño, el perfilamiento de usuarios se estructuró a través de tres técnicas de recolección cualitativa en el contexto de la Escuela de Computación (UCV):

- **1. Observación Directa Participante:** Registro de patrones de interacción y dificultades en sesiones prácticas de laboratorio durante las primeras semanas del período académico I-2026.
- **2. Entrevistas Semiestructuradas Exploratorias:** Aplicadas a una muestra inicial de 6 estudiantes de pregrado que cursaban o habían cursado recientemente asignaturas con proyectos grupales (Ingeniería de Software), indagando sobre sus mayores temores y puntos de quiebre con Git.
- **3. Consulta Diagnóstica a Docentes de la Especialidad:** Intercambio cualitativo con profesores de desarrollo de software para identificar los errores más destructivos que cometen los equipos en sus repositorios de entrega.

5.2. Arquetipo Primario: Carlos Mendoza (El Estudiante Abrumado)

- **Datos Demográficos:** 21 años, estudiante del 4to semestre de la Licenciatura en Computación, UCV. Desarrollador junior autodidacta.

- **Contexto Académico:** Domina la lógica de algoritmos y programación estructurada/POO, pero carece de entrenamiento formal en flujos de trabajo colaborativos y herramientas distribuidas de control de versiones.
- **Trabajos del Usuario (Jobs to be Done):** Aprobar la asignatura de desarrollo colaborativo; integrar su código al repositorio del equipo sin colapsar el proyecto; dominar las herramientas estándar para construir un portafolio técnico.
- **Frustraciones y Dolores (Pains):** Terror paralizante a cometer un error irreparable (ej. sobreescritura accidental o merge fallido); desorientación espacial ante la falta de feedback visual en la terminal; confusión conceptual ante mensajes de conflicto con marcadores crípticos (<<<<<<< HEAD); documentación densa exclusivamente en inglés.
- **Ganancias Esperadas (Gains):** Espacio seguro de simulación (sandbox) donde equivocarse no destruya archivos reales; comprensión visual del movimiento de commits y punteros; explicaciones claras en español cuando ocurra una colisión; sensación de progreso gamificado.

5.3. Arquetipo Secundario: Profa. Elena Valero (La Docente Facilitadora)

- **Datos Demográficos:** 42 años, Profesora Agregada de la Escuela de Computación (UCV), docente de asignaturas de Ingeniería de Software y Proyectos de Programación.
- **Contexto Docente:** Coordina equipos de 4 a 6 estudiantes que deben desarrollar software integral durante el semestre. Cuenta con escaso tiempo en el plan curricular para dedicar clases enteras a la sintaxis básica de Git.
- **Trabajos del Usuario:** Asegurar que los equipos estudiantiles trabajen de forma colaborativa ordenada y sin dependencias bloqueantes; evaluar la trazabilidad y contribución real de cada alumno en el historial de commits.

- **Frustraciones y Dolores:** Invertir horas valiosas de asesoría técnica reparando repositorios estudiantiles corrompidos o desalineados en lugar de revisar diseño y arquitectura; excusas recurrentes de alumnos que «perdieron el código por culpa de Git»; herramientas existentes en inglés y desarticuladas del currículo.
- **Ganancias Esperadas:** Disponer de un recurso didáctico interactivo de libre acceso para asignar como nivelación previa a los proyectos grupales; constatar que los estudiantes adquieran autonomía conceptual en resolución de ramas y conflictos; reducción de incidencias en entregas evaluadas.

5.4. Mapa de Viaje del Usuario (User Journey Map)

A continuación se modela el flujo de experiencia de Carlos Mendoza al interactuar con el recurso propuesto:

Tabla 4

Mapa de Viaje del Usuario (Carlos Mendoza).

Etapa del Flujo	Acciones del Usuario	Puntos de Contacto	Pensamientos y Emociones	Puntos de Dolor	Oportunidad Didáctica en RED
1. Descubrimiento	Accede al enlace del RED sugerido por la Profa. Elena en el aula virtual de la UCV.	Enlace en Classroom / Moodle institucional.	Curioso, pero escéptico. «Otra guía teórica aburrida que tendré que leer».	Miedo a que requiera configuraciones complejas o instalaciones pesadas.	Carga ultra-rápida en el navegador, sin registro forzoso y mensaje de bienvenida motivador.
2. Onboarding	Explora la interfaz interactiva e inicia el primer módulo sin necesidad de cuenta.	Pantalla de bienvenida y selector de nivel inicial.	Aliviado. «No me pide instalar nada en la máquina del laboratorio».	Incertidumbre sobre qué nivel tomar.	Test diagnóstico rápido y visual de 3 preguntas para recomendar nivel de inicio.
3. Aprendizaje Core	Ingresa comandos de Git local (init, add, commit) en la terminal simulada.	Terminal web, panel de estados y Grafo Vivo.	Entusiasmado y enfocado. «¡Por fin veo adónde va el commit cuando hago add!».	Duda si escribe un comando con sintaxis incorrecta.	Feedback inmediato de sintaxis en español y animación fluida del nodo en el grafo.

Etapa del Flujo	Acciones del Usuario	Puntos de Contacto	Pensamientos y Emociones	Puntos de Dolor	Oportunidad Didáctica en RED
4. Conflicto Guiado	En el módulo de conflicto, fusiona dos ramas que modificaron la misma línea.	Consola y editor visual de conflicto interactivo.	Ansioso y tenso. «Este es el error que siempre me daña el proyecto».	Bloqueo cognitivo ante marcadores crípticos de colisión de código.	Interfaz limpia de comparación línea por línea con resaltado en colores contrastados.
5. Andamiaje Nova	Tras 90 segundos de inactividad, se activa la sugerencia socrática de la tutora Nova.	Panel flotante de tutoría y nodos resaltados.	Comprendido y empoderado. «Me explica por qué chocó, no solo qué botón tocar».	Riesgo de que la IA le dé la respuesta directa sin que aprenda.	Pistas heurísticas en forma de preguntas guía que orientan la deducción autónoma.
6. Consolidación	Resuelve la colisión, visualiza el commit de fusión integrado y completa el reto.	Pantalla de logro superado y resumen conceptual.	Orgulloso, seguro y con autoeficacia. «Ya no le tengo miedo a un merge en equipo».	Deseo de demostrar su aprendizaje al grupo y docente.	Insignia de logro descargable con resumen de conceptos dominados.

6. Análisis de la Competencia (Benchmarking)

Para determinar la propuesta de valor diferencial de «Aprende Git Interactivo», se realizó un análisis comparativo frente a soluciones existentes en el ecosistema internacional. Para subsanar declaraciones genéricas de accesibilidad, las alternativas se evaluaron bajo criterios técnicos concretos (inspección de código DOM, soporte de tecnologías asistivas y pruebas preliminares con herramientas de auditoría Lighthouse y WAVE):

- **Competidor Directo 1: Learn Git Branching (Cottle, s.f.).** Es el referente más extendido a nivel global. Destaca por su visualizador de grafos y terminal interactiva. Deficiencias técnicas verificadas: carece de soporte de accesibilidad digital demostrable (el lienzo SVG carece de etiquetas WAI-ARIA, los nodos no son navegables mediante teclado ni poseen descripciones de texto alternativas para lectores de pantalla, y el contraste cromático de ciertas ramas es menor a 3:1); no ofrece tutoría explicativa contextual ante errores; y la traducción al español contiene incoherencias sintácticas.

- **Competidor Directo 2: Oh My Git! (Wieluch y Morr, s.f.).** Videojuego de código abierto con excelente representación gráfica del repositorio. Deficiencias verificadas: al ser una aplicación de escritorio compilada (requiere descarga e instalación), resulta impracticable en computadoras institucionales con privilegios restringidos; carece por completo de accesibilidad para personas con discapacidad visual al renderizarse en motor gráfico no compatible con tecnologías asistivas nativas.
- **Competidores Indirectos: Clientes Gráficos de Producción (GitKraken, GitHub Desktop, Sourcetree).** Herramientas de entorno laboral que resuelven operaciones mediante botones, pero no poseen fines pedagógicos: el estudiante ejecuta acciones sin comprender el modelo mental subyacente, fomentando la dependencia instrumental.
- **Competidores de Contenido Pasivo: Cursos en Video y Documentación Oficial (Chacon y Straub, 2014; Git Project, s.f.).** Tutoriales lineales y documentación estática de alto contenido teórico con nula interactividad práctica, propensos a la fatiga cognitiva y abandono prematuro.

Tabla 5

Matriz de Benchmarking Competitivo con criterios técnicos.

Criterio Evaluado	Aprende Git Interactivo (Propuesta UCV)	Learn Git Branching (Web)	Oh My Git! (Juego Escritorio)	Clientes Gráficos (GitKraken)	Git-SCM (Doc Oficial)	Cursos en Video (Udemy/YouTube)
Simulación de Grafo Vivo	Sí, dinámico y reactivo en tiempo real.	Sí, visual en árbol interactivo.	Sí, representación gráfica en juego.	Sí, lectura de repositorio real local.	No, solo diagramas estáticos ASCII/PNG.	No, grabaciones pregrabadas fijas.
Resolución Guiada de Conflictos	Sí, editor interactivo línea por línea con colores y contexto.	No, resolución abstracta mediante comandos.	No, resolución automática simplificada.	Sí, interfaz de merge industrial no didáctica.	No, instrucciones puramente textuales.	No, demostración pasiva del instructor.

Criterio Evaluado	Aprende Git Interactivo (Propuesta UCV)	Learn Git Branching (Web)	Oh My Git! (Juego Escritorio)	Clientes Gráficos (GitKraken)	Git-SCM (Doc Oficial)	Cursos en Video (Udemy/YouTube)
Tutoría Cognitiva y Andamiaje	Sí, tutora Nova con pistas socráticas basadas en reglas.	No, únicamente mensajes de error de Git.	No, mensajes lúdicos preestablecidos.	No, diseñado para profesionales.	No, carece de feedback interactivo.	No, sin interactividad adaptativa.
Criterios de Accesibilidad Digital	Meta de diseño: pautas WCAG 2.2 AA y principios DUA en español.	Deficiente: sin navegación por teclado en grafo ni ARIA en SVG.	No accesible: aplicación compilada sin soporte lector pantalla.	Parcial: modos alto contraste, pero baja usabilidad asistiva.	Parcial: lectores de pantalla leen texto, pero no diagramas.	Deficiente: dependiente de subtítulos automáticos en video.
Disponibilidad y Costo de Acceso	Web gratuita, sin registro forzoso ni instalación.	Web gratuita en línea (cero costo).	Requiere instalación de binario local.	Freemium / Licencias de pago comercial.	Web gratuita abierta.	Modelos de suscripción o pago por curso.
Idioma y Contexto Cultural	Nativo en español, contextualizado a necesidades UCV.	Inglés (traducciones voluntarias parciales).	Inglés prioritario.	Multi-idioma técnico.	Multi-idioma formal.	Variable según creador de contenido.

7. Propuesta de Solución, Justificación Pedagógica y Arquitectura

7.1. Justificación Pedagógica y de Diseño Instruccional

El recurso «Aprende Git Interactivo» se cimenta sobre sólidas teorías del aprendizaje contemporáneo y el diseño de software educativo:

- **Constructivismo y Modelado Mental (Novak y Gowin, 1988):** El aprendizaje del control de versiones no puede ocurrir mediante memorización pasiva. Al manipular el Grafo Vivo, el estudiante autoconstruye el modelo mental de Git como un conjunto inmutable de instantáneas conectadas por referencias dirigidas, transformando conceptos invisibles en entidades espaciales manipulables.
- **Metas de Accesibilidad Universal y Principios DUA (Álvarez González et al., 2014; CAST, 2018; World Wide Web Consortium, 2023):** Se formula como meta de diseño el cumplimiento de las pautas WCAG 2.2 nivel AA y los principios del Diseño Universal para

el Aprendizaje. Esto se traduce en: (a) Múltiples formas de representación: el estado del repositorio se comunica simultáneamente en texto estructurado en consola, tabla semántica accesible y grafo visual con doble codificación (forma + color); (b) Múltiples formas de acción: navegación integral por teclado con foco visible gestionado; y (c) Múltiples formas de motivación: progreso dosificado con niveles de dificultad adaptativa.

- **Aprendizaje Significativo y Andamiaje (Díaz-Barriga Arceo y Hernández Rojas, 2002; Woolf, 2009):** El andamiaje instruccional se activa dinámicamente cuando el alumno se enfrenta a situaciones de conflicto o bloqueo, evitando que la frustración degenere en abandono.
- **Interacción Persona-Computadora y Pensamiento Computacional (Lodi y Martini, 2021; Shneiderman y Plaisant, 2005):** Diseño de interfaz con retroalimentación visual continua y reducción de carga cognitiva, permitiendo al estudiante razonar en términos de abstracciones computacionales claras.

7.2. Arquitectura de la Tutoría Inteligente (Nova): Especificación y Salvaguardas

En atención a los requerimientos de rigor y viabilidad, se detalla el funcionamiento de la tutora pedagógica Nova:

- **Arquitectura Técnica en el MVP:** En el Producto Mínimo Viable, Nova opera a través de un motor de reglas heurísticas e inferencia determinista en el cliente (JavaScript). Analiza el estado JSON del árbol de Git (ramas existentes, posición del puntero HEAD, archivos en staging) y el último comando ingresado, contrastándolo con el objetivo pedagógico del nivel.
- **Datos Procesados y Privacidad Estricta:** Nova procesa exclusivamente el estado abstracto de la simulación (nombres de ramas de ejercicio, identificadores hash de commits simulados y mensajes de error estándar). No recopila nombres, direcciones IP, código fuente personal ni

datos sensibles, aplicando el principio de minimización de datos. Si se activa la pasarela opcional a un modelo de lenguaje, solo se envían la pregunta del estudiante y ese estado abstracto de la simulación.

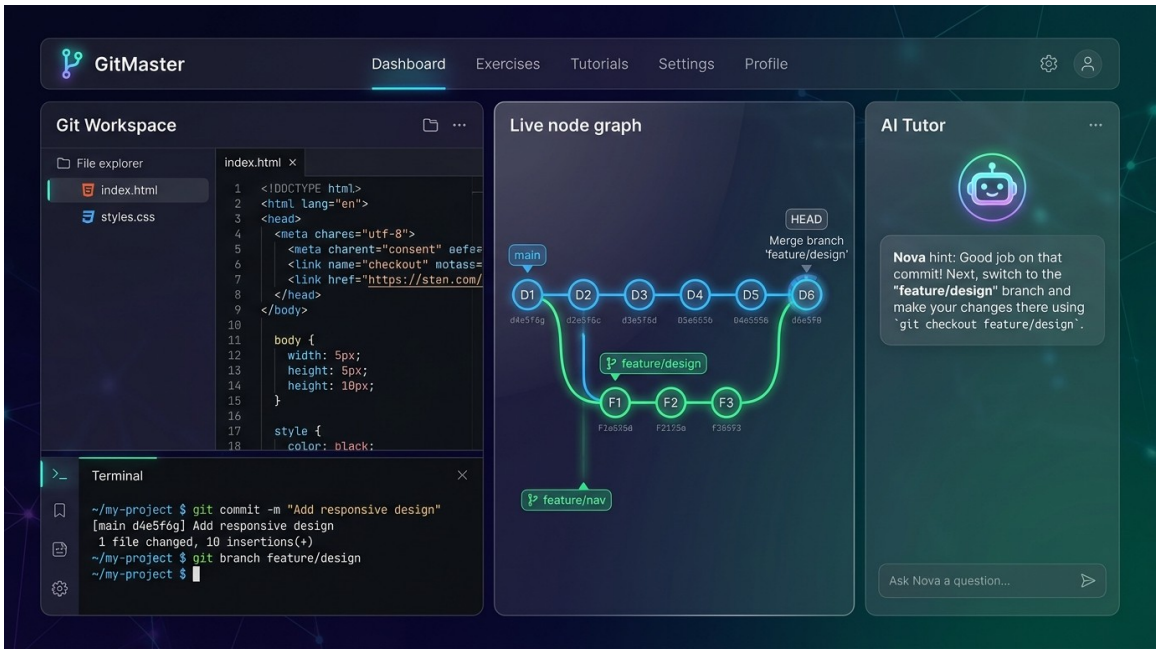
- **Prevención de Alucinaciones y Enfoque Socrático:** A diferencia de un chat generativo abierto que podría inventar comandos o entregar la respuesta resuelta privando al alumno del proceso de aprendizaje, Nova está restringida por salvaguardas (guardrails): emite pistas en tres niveles de andamiaje: (1) Pregunta reflexiva («Observa a qué rama apunta tu puntero HEAD actualmente»); (2) Pista analítica («Para fusionar los cambios de 'feature' en 'main', debes situarte primero en la rama receptora»); y (3) Orientación sintáctica («Revisa la documentación del comando checkout o switch»).

7.3. Prototipo Conceptual de Interfaz (Mockup)

A continuación se presenta el diseño preliminar de la interfaz del RED en modo de alto contraste oscuro, optimizada para minimizar la carga cognitiva del usuario:

Figura 1

Mockup conceptual de la plataforma «Aprende Git Interactivo» en modo oscuro accesible. Distribución tripartita: editor/terminal a la izquierda, Grafo Vivo interactivo al centro y panel de tutoría socrática Nova a la derecha.



8. Encaje de Valor (Value Proposition Canvas)

Para verificar la correspondencia unívoca entre las necesidades del perfil de usuario y la arquitectura de la solución, se estructura el encaje de valor a través de una matriz integral que relaciona cada trabajo, dolor y ganancia con su respectivo producto, aliviador o creador de ganancia:

Tabla 6

Matriz Integral de Encaje de Valor (Correspondencia 1 a 1).

Dimensión del Usuario (Carlos / Docente)	Propuesta de Valor del RED (Solución)	Mecanismo de Correspondencia y Validación
Trabajo 1: Aprender a usar Git para coordinar proyectos de asignaturas colaborativas.	Producto: Entorno web de simulación con terminal guiada y ejercicios de flujo grupal.	Correspondencia directa: Los ejercicios replican escenarios reales de asignaturas de la UCV.

Dimensión del Usuario (Carlos / Docente)	Propuesta de Valor del RED (Solución)	Mecanismo de Correspondencia y Validación
Trabajo 2: Comprender qué ocurre internamente al fusionar ramas de código.	Producto: Grafo Vivo reactivo que dibuja commits y punteros en tiempo real.	Correspondencia directa: Transforma la memoria sintáctica ciega en comprensión topológica espacial.
Dolor 1 (Terror al error): Miedo a ejecutar comandos destructivos y borrar código del equipo.	Aliviador: Entorno sandbox en el navegador, sin consecuencias sobre archivos reales.	Mitigación de dolor: Permite experimentar y equivocarse libremente construyendo resiliencia.
Dolor 2 (CLI Anxiety): Bloqueo cognitivo ante pantallas negras de consola y mensajes crípticos.	Aliviador: Consola amigable con mensajes explicativos en español y feedback visual sincronizado.	Mitigación de dolor: Reduce la barrera lingüística y la carga cognitiva de la sintaxis críptica.
Ganancia 1 (Autonomía): Resolver colisiones de código por cuenta propia sin depender de otros.	Creador de Ganancia: Simulador interactivo de conflictos línea por línea con pistas socráticas Nova.	Generación de valor: Desarrolla autoeficacia técnica y criterio profesional en el estudiante.
Ganancia 2 (Autoeficacia y Motivación): Sentir un progreso palpable y validado.	Creador de Ganancia: Sistema de logros gamificados e insignias de dominio de conceptos clave.	Generación de valor: Refuerza la persistencia académica y previene la deserción temprana.

PARTE II — FASE II: DISEÑO INSTRUCCIONAL, DISEÑO INFORMÁTICO Y DESARROLLO DEL PROTOTIPO

Esta parte documenta el proceso de desarrollo del Producto Mínimo Viable (MVP) del Recurso Educativo Digital, denominado **GitPlayground** (Torres et al., 2026), desplegado públicamente en https://git-playground-ucv.netlify.app y con código fuente versionado en https://github.com/ShonnyAIO/git-playground-drip. Responde a los elementos de diseño instruccional (puntos 4 a 7) y de diseño informático (puntos 8 a 12) solicitados para la Fase II.

9. Diseño Instruccional

9.1. Competencias y Habilidades que el RED Ayuda a Desarrollar

Las competencias se formulan como resultados de aprendizaje observables, siguiendo la taxonomía revisada de Bloom (Anderson y Krathwohl, 2001). Cada una se asocia con el componente del RED donde se ejercita y con la evidencia que permite verificarla, lo que mantiene la trazabilidad entre el problema diagnosticado (sección 1), el perfil del estudiante (sección 5) y las funcionalidades del prototipo.

Tabla 7

Competencias específicas, nivel cognitivo y evidencia en el RED.

Competencia (el estudiante será capaz de…)	Nivel (Bloom)	Dónde se ejercita en el RED	Evidencia de logro
C1. Explicar el ciclo de vida de un archivo entre Working Directory, Staging Area y repositorio local.	Comprender	Lección 1; Simulador Core (zonas de estado).	Archivos preparados y confirmados correctamente; medallas «Repositorio Iniciado» y «Creador de Historias».
C2. Crear ramas, cambiar entre ellas e interpretar la posición del puntero HEAD en el grafo.	Aplicar	Lección 2; Simulador Core, Misión 1.	Rama creada y commit aislado; medalla «Explorador de Ramas».
C3. Integrar ramas con git merge y distinguir una fusión fast-forward de una de tres vías.	Analizar	Lección 3; Simulador Core, Misión 1.	Fusión completada en el grafo; medalla «Maestro del Merge».

Competencia (el estudiante será capaz de…)	Nivel (Bloom)	Dónde se ejercita en el RED	Evidencia de logro
C4. Valorar cuándo conviene rebase o merge y reconocer el riesgo de reescribir historia publicada.	Evaluar	Lección 4; Conflictos (teoría Merge vs Rebase); Quiz.	Respuesta correcta en el quiz y conflicto resuelto en modo rebase.
C5. Sincronizar un repositorio local con uno remoto y colaborar mediante Pull Requests y revisión de código.	Aplicar	Lección 5; GitHub & Remotos.	Secuencia remote → push → PR → review → merge → pull completada; medalla «Enlazador de Nube».
C6. Resolver un conflicto de fusión leyendo los marcadores y decidiendo la versión final.	Aplicar / Crear	Lección 6; Resolución de Conflictos.	Conflicto marcado como resuelto y commit de fusión creado; medalla «Domador de Conflictos».
C7. Recuperarse de errores con git restore, git reset y git revert.	Aplicar	Simulador Core, Misión 2.	Misión 2 completada; medalla «Viajero del Tiempo».
C8. Seleccionar un flujo de trabajo de equipo (Gitflow o trunk-based) adecuado a un caso.	Evaluar	Desafíos & Quizzes.	Cinco respuestas correctas; medalla «Sabio de Git».

Competencias transversales: (a) autorregulación del aprendizaje, mediante el progreso visible (XP, niveles y lecciones completadas) y la posibilidad de repetir pasos a ritmo propio; (b) lectura e interpretación de mensajes técnicos en la terminal, con explicaciones en español; y (c) disposición al trabajo colaborativo en equipos de software, al practicar en un entorno seguro los flujos que luego se usan en asignaturas grupales.

9.2. Teoría o Paradigma de Enseñanza y Aprendizaje

El RED se fundamenta en el **paradigma constructivista**: el estudiante construye el modelo mental de Git actuando sobre un repositorio simulado y observando las consecuencias de cada comando, en lugar de memorizar sintaxis (Novak y Gowin, 1988; Díaz-Barriga Arceo y Hernández Rojas, 2002). Este paradigma se concreta mediante cuatro marcos complementarios, cada uno con una decisión de diseño verificable en el prototipo:

- **Zona de desarrollo próximo y andamiaje (Vygotsky, 1978; Wood et al., 1976):** la tutora Nova ofrece ayuda graduada —pregunta reflexiva, pista analítica y orientación sintáctica— que se retira a medida que el estudiante gana autonomía, sin entregar la respuesta resuelta.
- **Teoría cognitiva del aprendizaje multimedia (Mayer, 2021):** las lecciones animadas aplican los principios de segmentación (pasos cortos que el estudiante avanza a su ritmo), señalización (resaltado del elemento que el subtítulo nombra), modalidad y redundancia controlada (subtítulos siempre disponibles y narración por voz opcional) y contigüidad (el comando aparece junto a su efecto en el grafo).
- **Teoría de la carga cognitiva (Sweller, 1988):** cada escena muestra solo las zonas necesarias para el concepto en curso; la terminal simulada traduce la abstracción en cambios visibles inmediatos, reduciendo la carga extrínseca de interpretar una consola sin retroalimentación.
- **Aprendizaje experiencial (Kolb, 1984) y Diseño Universal para el Aprendizaje (CAST, 2018):** la secuencia observar → practicar → reflexionar → aplicar se repite por tema (Figura 2), y el contenido se ofrece por múltiples medios de representación (texto, animación, voz), de acción (ratón, teclado, pantalla táctil) y de compromiso (misiones, medallas, progreso).

9.3. Temas, Lecciones y Secciones que Conforman el RED

El contenido se organiza en ocho temas de dificultad creciente. Cada tema combina, cuando aplica, una lección animada (modelo mental), una práctica interactiva (aplicación) y un mecanismo de evaluación o reconocimiento (verificación):

Tabla 8

Temas, lecciones y secciones del RED GitPlayground.

N.º	Tema	Lección animada (duración)	Práctica interactiva	Evaluación / reconocimiento	Nivel
1	Los tres estados de Git	Lección 1 (0:58)	Simulador Core: init, add, commit, status	Medallas «Repositorio Iniciado» y «Creador de Historias»	Básico
2	Ramas y HEAD	Lección 2 (0:49)	Simulador Core: branch, checkout (Misión 1)	Medalla «Explorador de Ramas»	Básico
3	Merge: fast-forward y de tres vías	Lección 3 (0:46)	Simulador Core: merge (Misión 1)	Medalla «Maestro del Merge»	Intermedio
4	Rebase frente a merge	Lección 4 (0:48)	Conflictos: teoría y flujo rebase --continue	Pregunta 2 del quiz	Avanzado
5	Remotos: push, fetch y pull	Lección 5 (0:41)	GitHub & Remotos: push, Pull Request, review, merge, pull	Medalla «Enlazador de Nube»	Intermedio
6	Anatomía de un conflicto	Lección 6 (0:46)	Resolución de Conflictos: editor de tres vías	Medalla «Domador de Conflictos»	Avanzado
7	Recuperación ante errores	— (práctica directa)	Simulador Core: restore, reset, revert (Misión 2)	Medalla «Viajero del Tiempo»	Intermedio
8	Flujos de trabajo profesionales	—	Desafíos & Quizzes: 5 casos (Gitflow, trunk-based, rebase, revert, detached HEAD)	Medalla «Sabio de Git»	Avanzado

Completar las seis lecciones otorga la medalla «Autodidacta Visual». La tutora Nova y el Dashboard (misión narrativa, nivel y ruta de aprendizaje) acompañan transversalmente a todos los temas. Las lecciones se verificaron contra la documentación oficial de Git (Chacon y Straub, 2014).

9.4. Tipo de Aprendizaje que se Potencia y Estrategias Utilizadas

El RED potencia un **aprendizaje activo, experiencial y significativo**, con descubrimiento guiado: el estudiante formula hipótesis («¿qué pasará con HEAD si hago commit aquí?»), las prueba en el simulador y contrasta el resultado con el modelo de la lección. Se

privilegia la comprensión del modelo de datos de Git (instantáneas enlazadas en un grafo dirigido) sobre la memorización de comandos. Las estrategias didácticas implementadas se resumen en la Tabla 9.

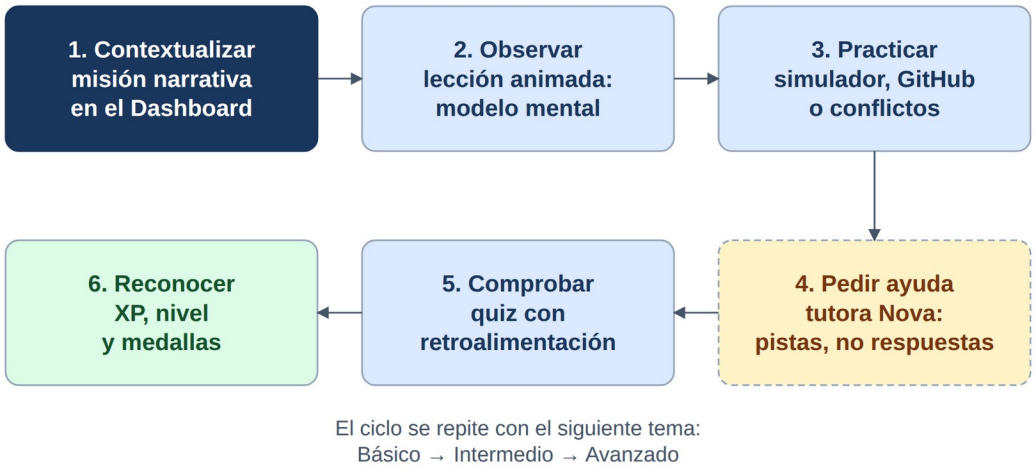
Tabla 9

Estrategias didácticas y su implementación en el prototipo.

Estrategia	Implementación en el RED	Fundamento
Contextualización narrativa	Misión «Rescate de código» en el Dashboard: el estudiante asume el rol de ingeniero que ordena el repositorio de su equipo.	Aprendizaje situado; motivación (CAST, 2018).
Demostración visual (ejemplos resueltos animados)	Seis lecciones en las que archivos, commits y ramas se desplazan entre estados; reproducción paso a paso y control de velocidad.	Mayer (2021): segmentación y señalización.
Práctica guiada en entorno seguro (sandbox)	Misiones con pasos requeridos en el Simulador, flujo de colaboración en GitHub & Remotos y editor de conflictos; ningún error afecta archivos reales.	Kolb (1984); reducción del miedo al error (sección 8).
Andamiaje socrático graduado	Tutora Nova con pistas en tres niveles según el estado del repositorio; preguntas sugeridas («¿Qué es HEAD?»).	Wood et al. (1976); Vygotsky (1978).
Retroalimentación inmediata	Mensajes de la terminal en español, actualización instantánea del grafo y explicación tras cada respuesta del quiz.	Hattie y Timperley (2007).
Gamificación	Puntos de experiencia, cinco niveles (de «Novato en Git» a «Maestro Git de la UCV») y nueve medallas vinculadas a logros reales.	Deterding et al. (2011).
Aprendizaje basado en casos	Conflicto real en App.jsx entre dos ramas; preguntas del quiz formuladas como situaciones de equipo.	Díaz-Barriga Arceo y Hernández Rojas (2002).
Representación múltiple y accesible	Subtítulos, narración por voz opcional, navegación completa por teclado, temas claro y oscuro, diseño adaptable a teléfonos.	CAST (2018); World Wide Web Consortium (2023).

Figura 2

Secuencia didáctica recomendada: ciclo que se repite en cada tema con dificultad creciente.



Nota. Elaboración propia.

10. Diseño Informático

10.1. Metodologías y Técnicas de Prototipado de Software

El desarrollo siguió un **prototipado evolutivo e incremental** (Sommerville, 2016): el MVP no se descarta, sino que se refina en iteraciones cortas hasta convertirse en el producto. Esta técnica se articuló con la etapa de Desarrollo del modelo ADDIE descrito en la sección 2. Concretamente se aplicaron:

- **Prototipado horizontal seguido de vertical:** la primera iteración construyó toda la navegación y las seis secciones con funcionalidad básica (prototipo horizontal); las siguientes profundizaron en funcionalidades completas, como el reproductor de lecciones animadas (prototipos verticales).
- **Bocetos de baja fidelidad antes del código:** cada pantalla se esbozó como wireframe con zonas numeradas (sección 10.4) y luego se implementó en alta fidelidad.

- **Desarrollo por incrementos verificables:** la tercera iteración se planificó como un plan de trece incrementos («slices»), cada uno con un criterio de aceptación, pruebas y evidencia visual antes de integrarse. Las decisiones tomadas quedaron registradas en el repositorio (carpeta specs/).
- **Contenido como datos y laboratorio de prototipos:** las lecciones son archivos de datos validados automáticamente; una página de laboratorio muestra cada lección como guion gráfico (Figura 4) para revisar el contenido sin recorrer la aplicación.
- **Verificación continua:** pruebas unitarias, pruebas de extremo a extremo con casos de uso (TUC), auditoría automatizada de accesibilidad y comparación de capturas contra una línea base para detectar regresiones visuales.

Tabla 10

Iteraciones del prototipo evolutivo (fechas tomadas del historial del repositorio Git).

Iteración	Fecha	Alcance	Técnica principal	Evidencia
1. Prototipo horizontal	30/05/2026	Estructura de navegación y seis secciones: Dashboard, Simulador, GitHub, Conflictos, Quizzes y tutora Nova.	Prototipado horizontal	Commits iniciales del repositorio.
2. Mejoras de UX	19/09/2026	Primer ciclo de mejoras de experiencia, Videoteca, pasarela opcional de IA para Nova y primera suite de pruebas de interfaz (7 casos).	Iteración evolutiva	Suite Playwright 7/7.
3. Lecciones animadas y base UX	03/10/2026	Seis lecciones animadas propias, progreso persistente, diseño adaptable a teléfonos y accesibilidad.	Prototipo vertical por incrementos	46 pruebas unitarias; 12 casos de interfaz; Pull Request #1.
4. Ajustes y documentación	10/10/2026	Correcciones del informe de la Fase I y saneamiento del código.	Revisión	Commit de correcciones.

10.2. Herramientas Seleccionadas para el Desarrollo del Prototipo

Se optó por desarrollo con código abierto en lugar de plataformas low-code o no-code. La razón principal es la naturaleza del contenido: simular Git exige un modelo de estados preciso (repositorio, ramas, HEAD, zonas) y animaciones sincronizadas con ese modelo. Además, el código abierto da control fino de la accesibilidad, permite alojamiento gratuito y deja el propio recurso versionado con Git, coherente con lo que enseña.

Tabla 11

Herramientas del prototipo: versión, sitio oficial y justificación de la selección.

Herramienta	Versión	Sitio oficial	Justificación de la selección
React (Meta Platforms, 2026)	19.2.6	https://react.dev	Interfaz por componentes y estado declarativo: cada comando actualiza el modelo y la vista se redibuja sola.
Vite (VoidZero, 2026a)	8.0.14	https://vite.dev	Servidor de desarrollo con recarga instantánea y empaquetado optimizado para producción.
JavaScript, CSS y SVG	Estándar web	https://developer.mozilla.org	SVG para el grafo de commits y las lecciones animadas; CSS para transiciones y temas claro/oscuro.
Lucide React	1.17.0	https://lucide.dev	Iconografía abierta y consistente.
Web Speech API (Mozilla Developer Network, s.f.)	Del navegador	https://developer.mozilla.org/es/docs/Web/API/Web_Speech_API	Narración opcional en español de los subtítulos, sin grabar audio.
Vitest (VoidZero, 2026b)	5.0.3	https://vitest.dev	Pruebas unitarias del estado del estudiante y del motor de lecciones.
Playwright (Microsoft, 2026)	1.57.0	https://playwright.dev	Pruebas de extremo a extremo que simulan a un estudiante usando la interfaz.
axe-core (Deque Systems, 2026)	4.13.0	https://github.com/dequelabs/axe-core	Auditoría automatizada de accesibilidad frente a WCAG.
ESLint	10.4.1	https://eslint.org	Análisis estático para mantener el código libre de errores comunes.
Git y GitHub	—	https://github.com	Control de versiones, revisión mediante Pull Requests e historial de iteraciones.
Netlify (Netlify, s.f.)	Plan gratuito	https://www.netlify.com	Alojamiento estático con publicación automática al integrar cambios en la rama principal.

Herramienta	Versión	Sitio oficial	Justificación de la selección
Node.js	26.7.0	https://nodejs.org	Entorno de ejecución para las herramientas de desarrollo.

Tabla 12

Ventajas y limitaciones observadas en las herramientas.

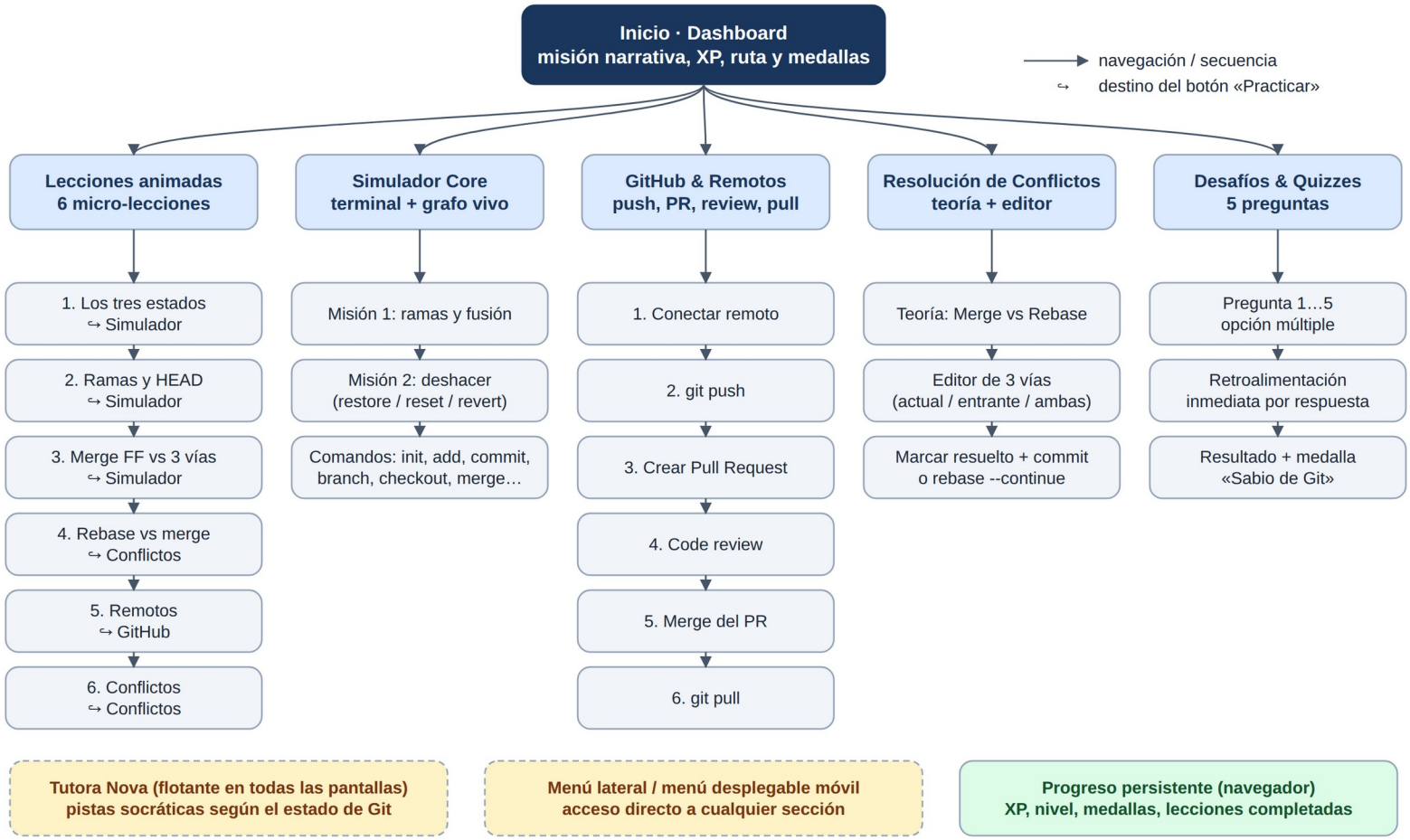
Herramienta	Ventajas observadas	Limitaciones observadas
React + Vite	Desarrollo rápido; compilación de producción en menos de medio segundo; paquete final de 102,9 kB comprimidos.	Requiere conocimientos de programación (no apto para docentes sin formación técnica); el estado vive en el navegador.
SVG y transiciones CSS	Animaciones fluidas sin bibliotecas adicionales; el texto se mantiene nítido a cualquier tamaño.	La animación de trazos curvos (propiedad d) solo es fluida en navegadores basados en Chromium.
Web Speech API	Voz en español sin costo ni archivos de audio.	La calidad y disponibilidad de la voz dependen del sistema operativo del estudiante.
Vitest, Playwright y axe-core	Detectaron regresiones reales antes de publicar (p. ej., un menú lateral deformado en escritorio).	La auditoría automática cubre solo una parte de los criterios WCAG; no sustituye la evaluación con usuarios.
Netlify	Publicación automática y gratuita desde GitHub; HTTPS incluido.	Sin servidor propio en el plan usado: el progreso se guarda solo en el navegador del estudiante.

10.3. Estructura del Recurso Didáctico: Secuencia de Pantallas, Enlaces e Interacción

El RED adopta una **estructura en red con centro (hub)**: el Dashboard articula cinco secciones a las que también se accede desde el menú lateral en todo momento. Dentro de cada sección la secuencia es lineal y guiada (pasos de una misión, pasos de una lección, preguntas del quiz), lo que combina libertad de exploración con andamiaje. Los botones «Practicar» de cada lección enlazan con la práctica correspondiente. La Figura 3 muestra el mapa de navegación y la Tabla 13 detalla, por pantalla, los elementos de enlace y de interacción.

Figura 3

Mapa de navegación del RED (diagrama tipo red).



Nota. Elaboración propia.

Tabla 13

Secuencia de pantallas, elementos de enlace y elementos de interacción.

Pantalla	Propósito	Elementos de enlace	Elementos de interacción
Dashboard	Contextualizar y orientar.	Botones «Ingresar al Workspace» y «Lecciones animadas»; ruta de módulos.	Lectura de nivel, XP y medallas.
Lecciones animadas	Construir el modelo mental.	Temario de 6 lecciones; botón «Practicar»; enlaces «Para profundizar».	Reproducir/pausar, pasos, velocidad, subtítulos, voz, teclado (Espacio, flechas).
Simulador Core	Aplicar comandos locales.	Selector de misiones; botones «git status» y «git log».	Terminal con historial, botones de comandos, zonas de archivos y grafo navegable.
GitHub & Remotos	Colaborar con un remoto.	Secuencia guiada de la misión.	Conectar remoto, push, crear PR, responder revisión, fusionar, pull.
Resolución de Conflictos	Resolver colisiones.	Pestañas Teoría / Simulador; botón al Simulador.	Elegir versión actual, entrante o ambas; marcar resuelto; commit o rebase --continue.
Desafíos & Quizzes	Comprobar comprensión.	Botón «Siguiente»; reinicio del quiz.	Selección de opción con retroalimentación inmediata.
Tutora Nova (global)	Andamiaje.	Disponible desde todas las pantallas.	Preguntas sugeridas y chat.

A nivel de cada lección, la estructura es un **guion gráfico (storyboard)**: una secuencia de escenas en la que cada paso declara su escena completa y su subtítulo. La Figura 4 presenta el guion gráfico de la Lección 1 tal como lo genera el laboratorio de prototipos.

Figura 4

Guion gráfico de la Lección 1 «Los tres estados de Git»: diez escenas con su comando y subtítulo.



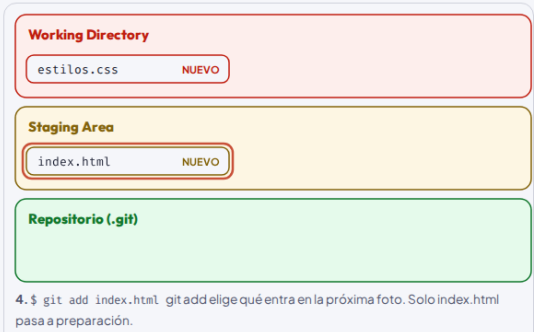
1. Git vigila tu proyecto en tres lugares: tu carpeta de trabajo, el área de preparación y el repositorio.



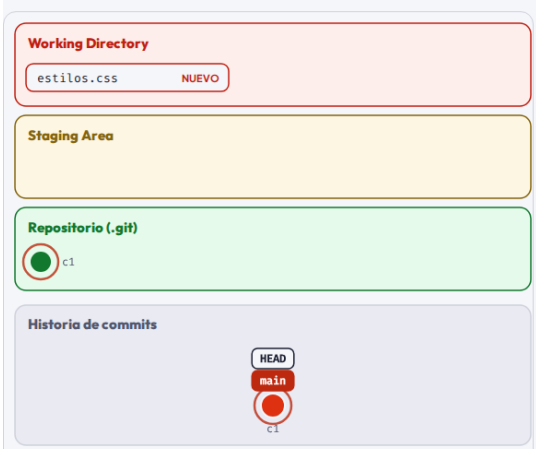
2. `$ git init` git init crea el repositorio: una carpeta oculta .git donde vivirá toda la historia.



3. Creas index.html y estilos.css. Git los ve, pero todavía no guarda nada de ellos.



4. `$ git add index.html` git add elige qué entra en la próxima foto. Solo index.html pasa a preparación.



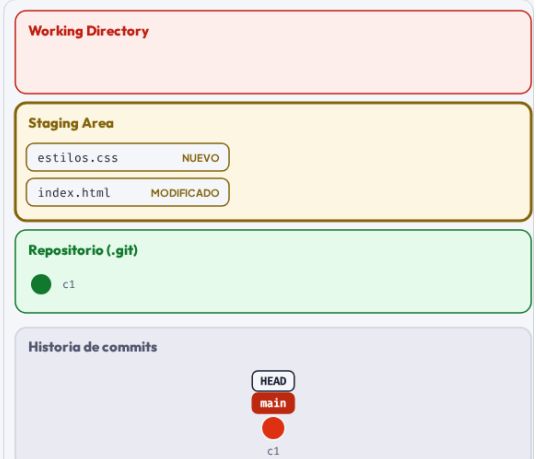
5. `$ git commit -m "Página inicial"` git commit toma la foto: nace el commit c1 con lo que estaba preparado.



6. estilos.css sigue en tu carpeta: lo que no preparas no entra en el commit.



7. Editas index.html. Git nota que cambió respecto al último commit y lo marca como



8. `$ git add .` git add . prepara todos los cambios de la carpeta de una vez.

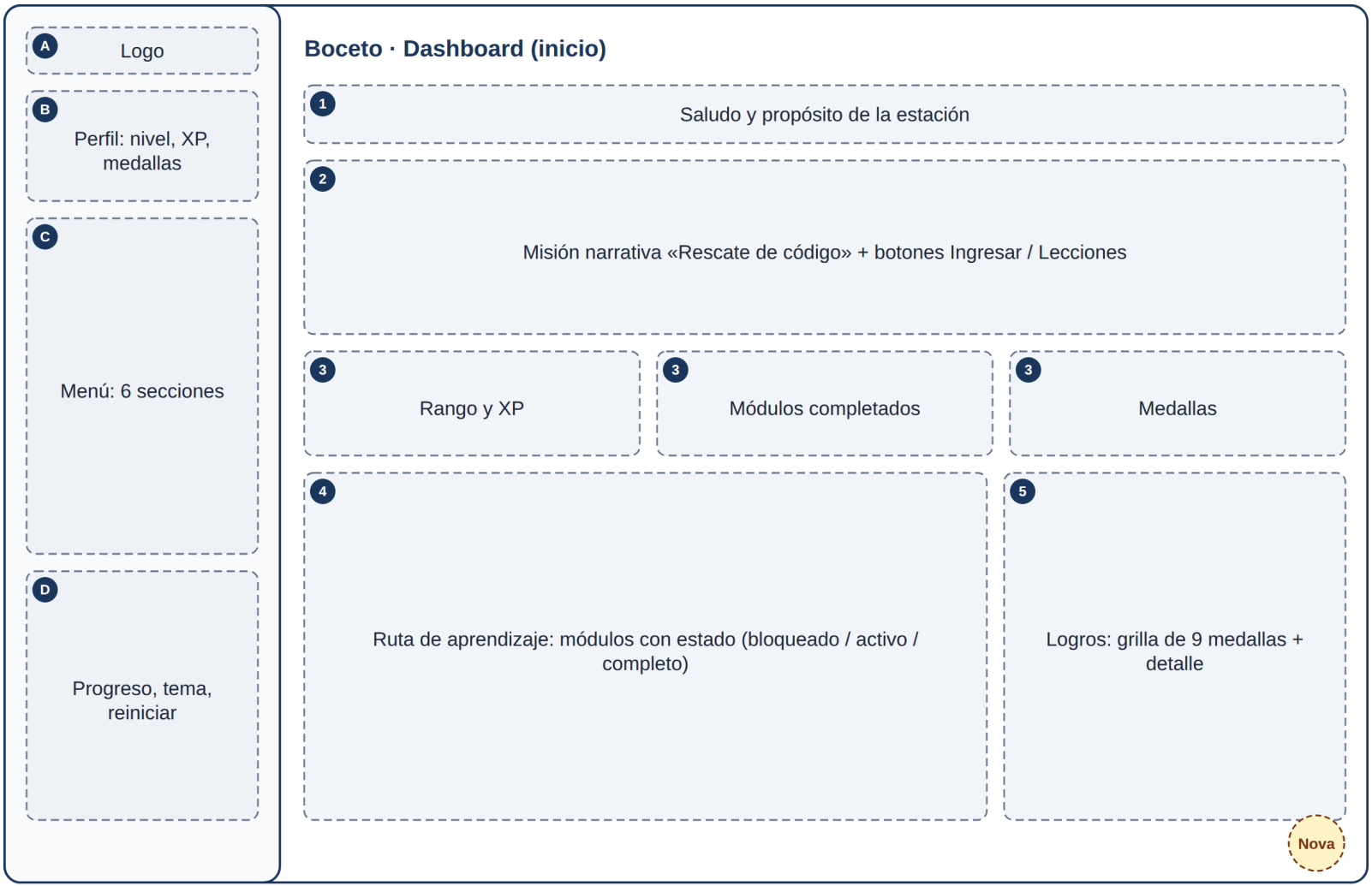
Nota. Elaboración propia.

10.4. Bocetos Generales de las Pantallas

Los bocetos (wireframes de baja fidelidad) definen la organización del contenido de cada vista antes de su implementación. Todas las pantallas de escritorio comparten un marco común: (A) logotipo, (B) perfil con nivel, XP y medallas, (C) menú de seis secciones, (D) progreso general, cambio de tema y reinicio, y la burbuja flotante de la tutora Nova. Las zonas numeradas de cada boceto corresponden al contenido propio de la pantalla. En teléfonos, el menú lateral se convierte en un menú desplegable (Figura 11).

Figura 5

Boceto del Dashboard: misión narrativa, indicadores, ruta de aprendizaje y logros.



Nota. Elaboración propia.

Figura 6

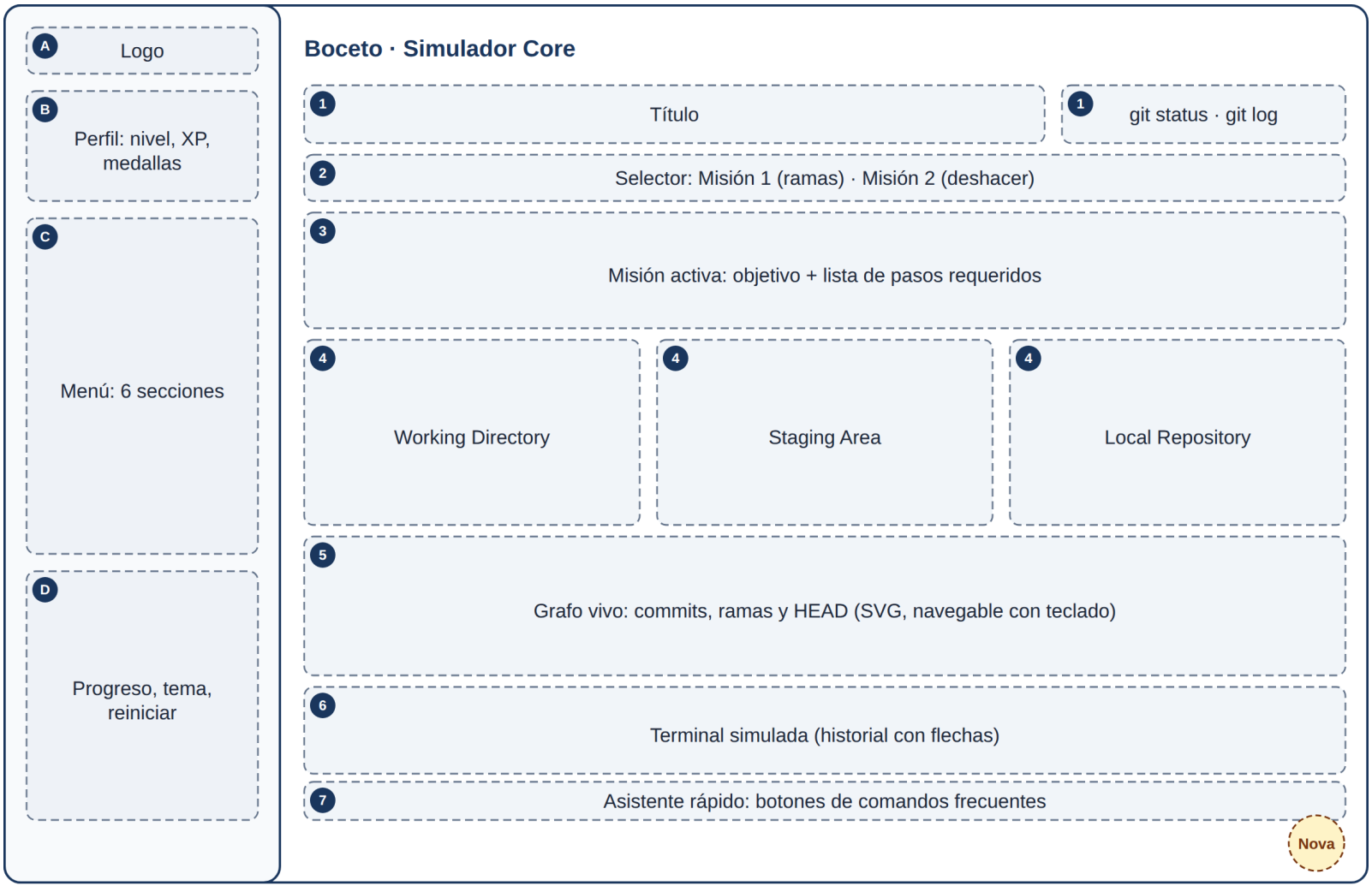
Boceto de Lecciones animadas: escenario, terminal y subtítulo, controles, ficha y temario.



Nota. Elaboración propia.

Figura 7

Boceto del Simulador Core: misión, tres zonas de estado, grafo vivo y terminal.



Nota. Elaboración propia.

Figura 8

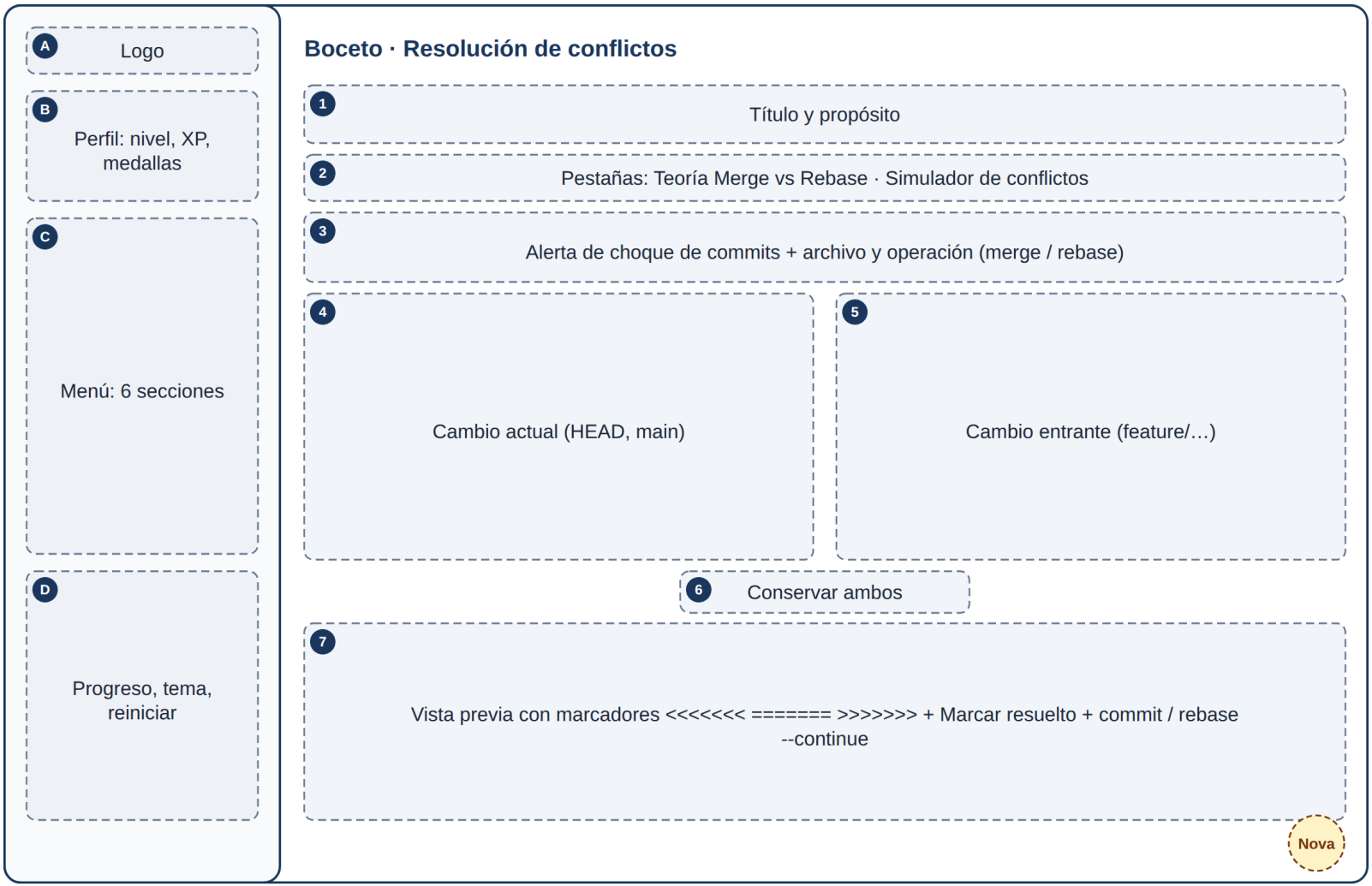
Boceto de GitHub & Remotos: repositorio local frente a servidor remoto simulado.



Nota. Elaboración propia.

Figura 9

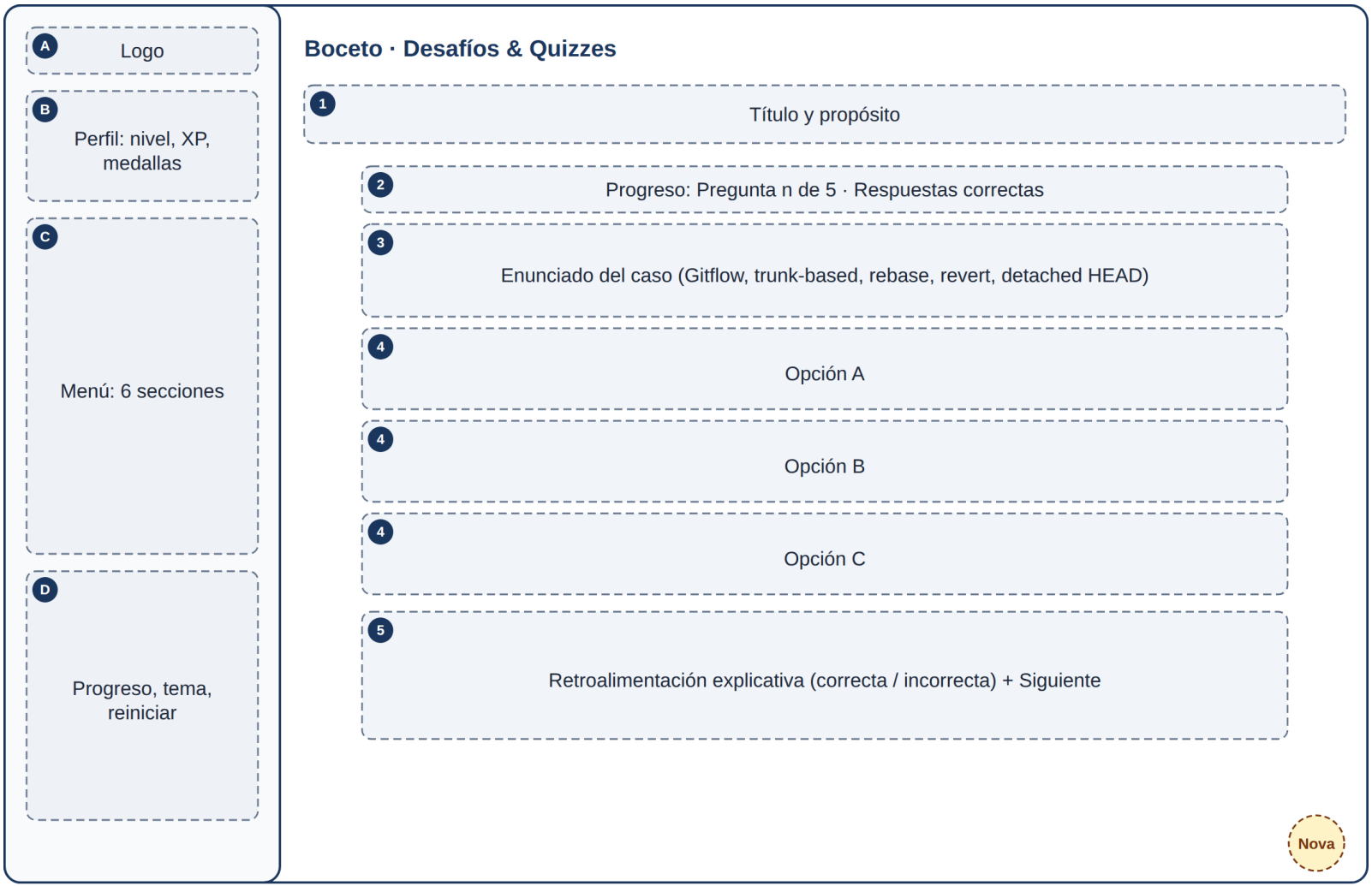
Boceto de Resolución de Conflictos: versiones en paralelo y vista previa con marcadores.



Nota. Elaboración propia.

Figura 10

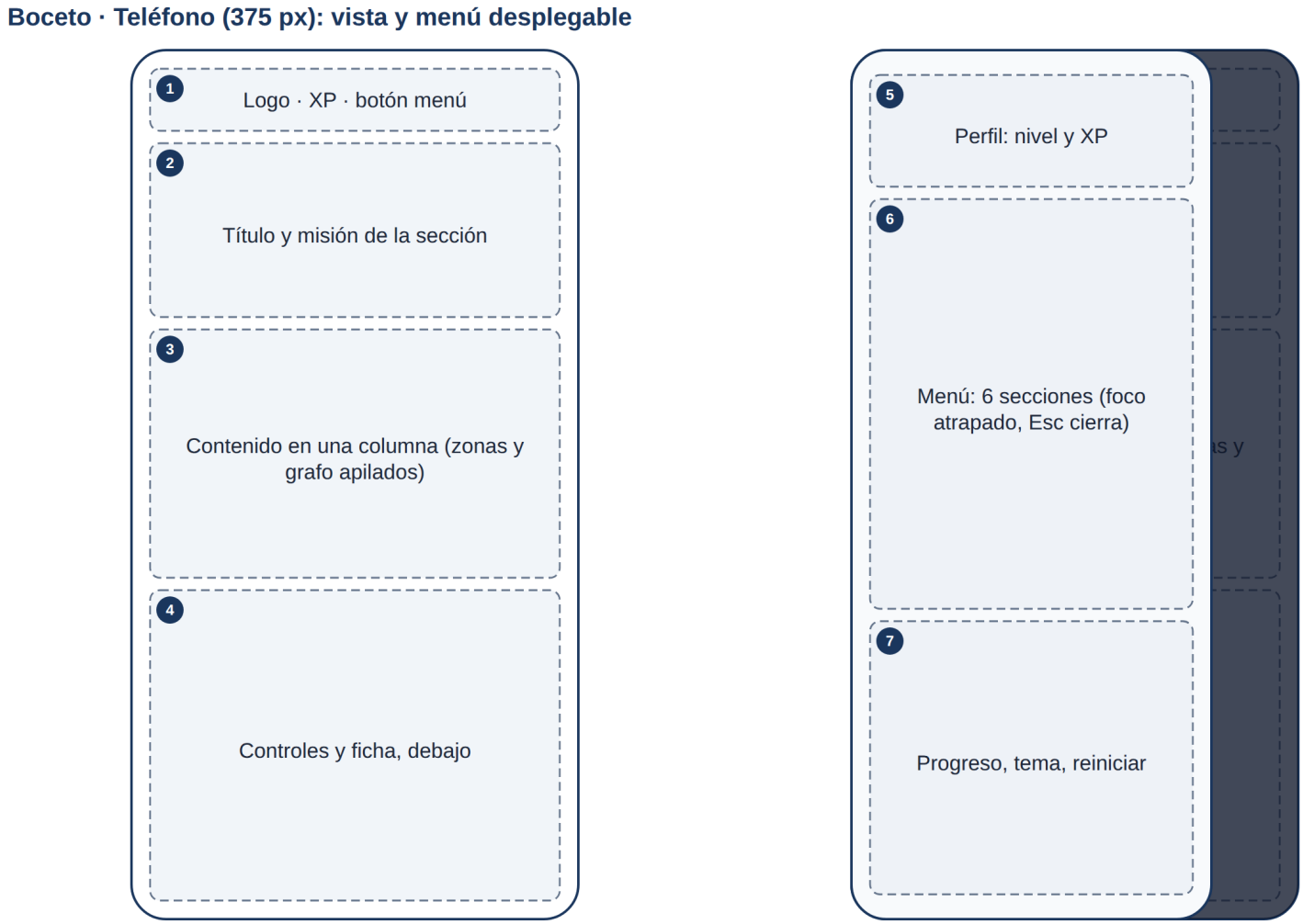
Boceto de Desafíos & Quizzes: enunciado, opciones y retroalimentación.



Nota. Elaboración propia.

Figura 11

Boceto en teléfono (375 px): contenido en una columna y menú desplegable con foco atrapado.

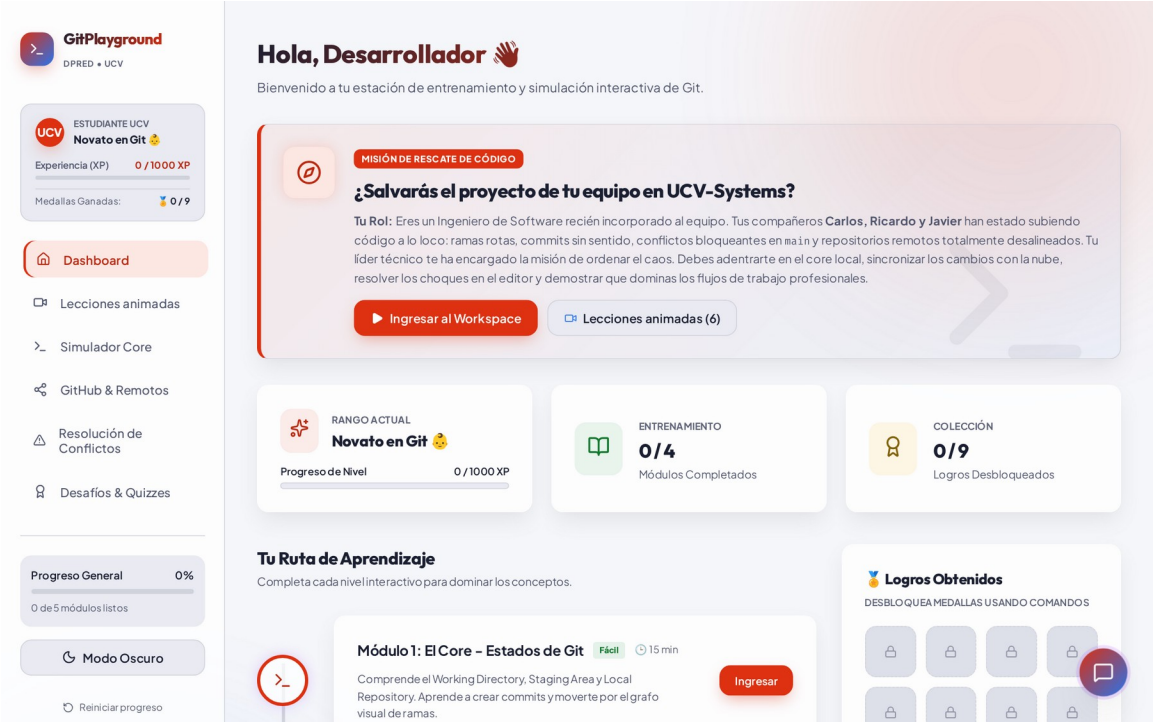


Nota. Elaboración propia.

Las Figuras 12 a 15 muestran las mismas vistas implementadas en alta fidelidad en el prototipo publicado (tema claro):

Figura 12

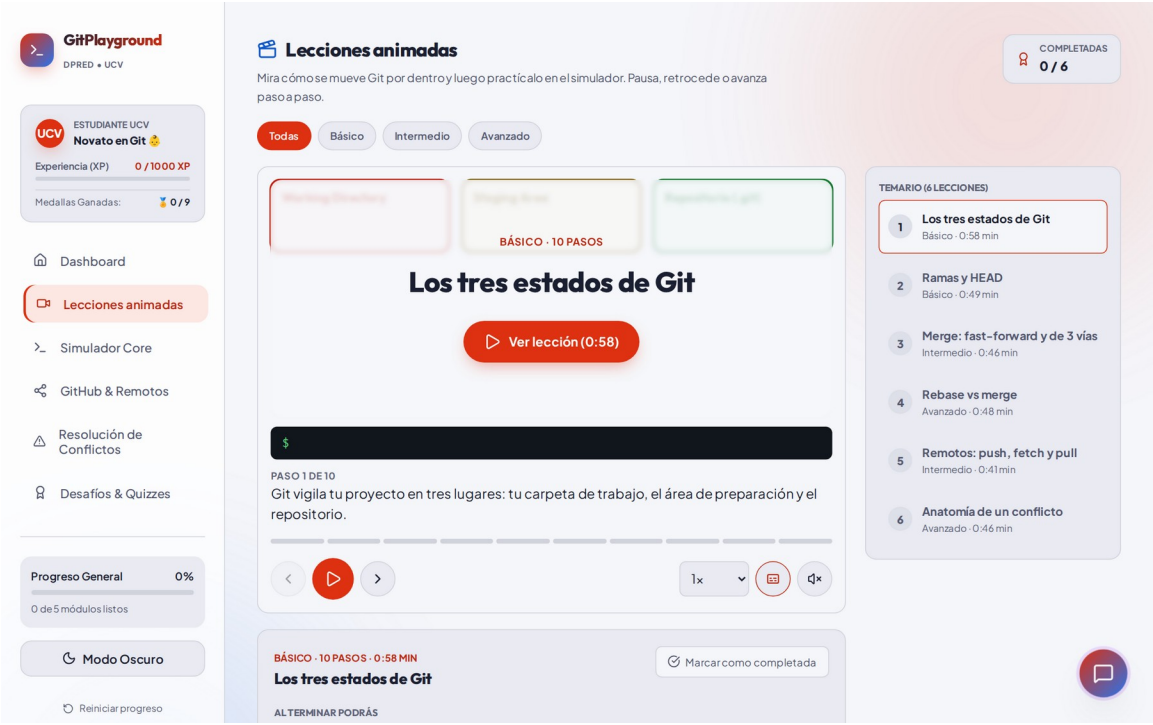
Prototipo: Dashboard.



Nota. Captura del prototipo publicado en https://git-playground-ucv.netlify.app..

Figura 13

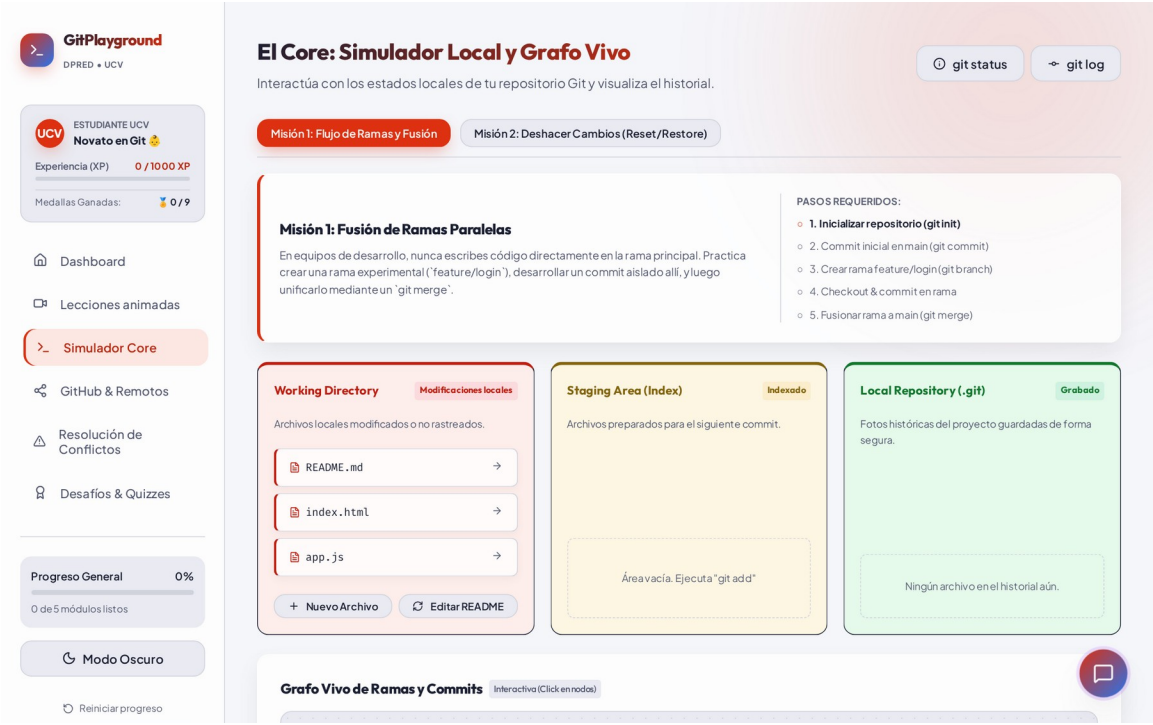
Prototipo: Lecciones animadas con el póster de la Lección 1.



Nota. Captura del prototipo publicado en https://git-playground-ucv.netlify.app..

Figura 14

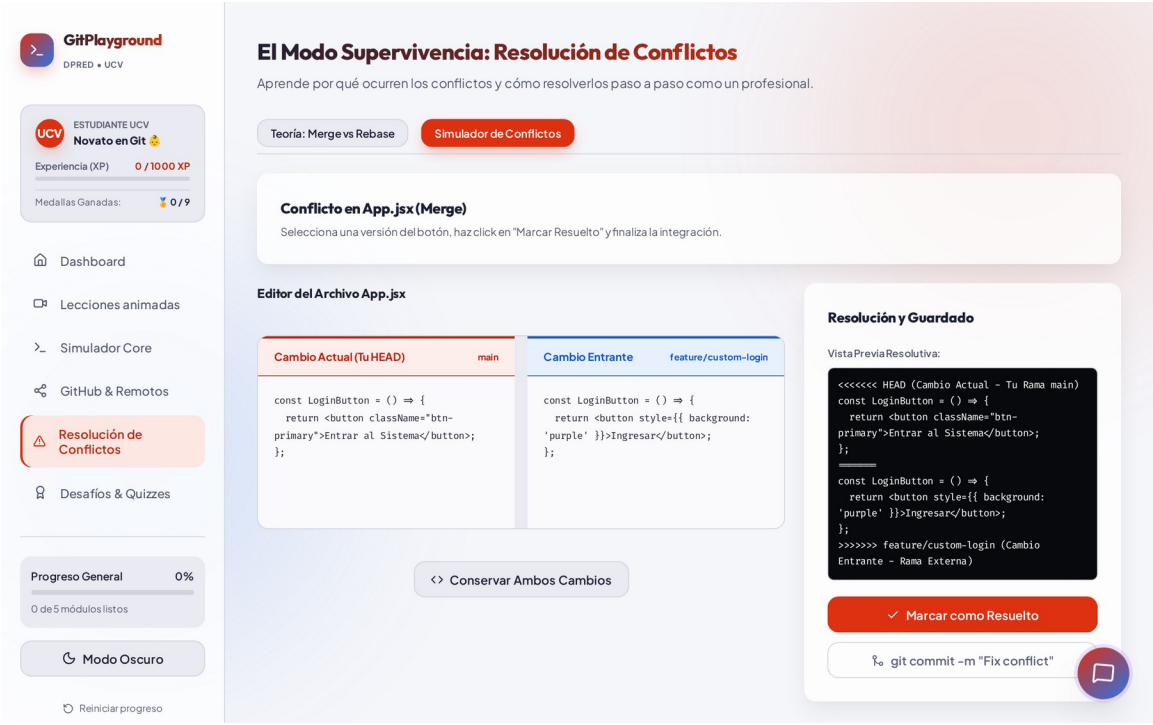
Prototipo: Simulador Core con la Misión 1.



Nota. Captura del prototipo publicado en https://git-playground-ucv.netlify.app..

Figura 15

Prototipo: editor de resolución de conflictos.



Nota. Captura del prototipo publicado en https://git-playground-ucv.netlify.app..

11. Validación y Correspondencia con el Prototipo Funcional

El diseño de las Fases I y II se contrastó con el prototipo funcional. El código, implementado en React 19 y Vite con arquitectura del lado del cliente (sin servidor propio), permite verificar la factibilidad técnica de los requerimientos. La Tabla 14 resume la trazabilidad entre requerimientos y componentes.

Tabla 14

Matriz de trazabilidad entre requerimientos de diseño y el prototipo.

Requisito / módulo	Componente en el código	Funcionalidad verificada	Estado y evidencia
Módulo 1: estados y grafo vivo	VisualSimulator.jsx	Terminal con init, status, add, commit, branch, checkout, merge, log, restore, reset y revert; tres zonas de estado; grafo SVG navegable con teclado.	Implementado; misiones 1 y 2 completables (casos de prueba TUC-UX-03).

Requisito / módulo	Componente en el código	Funcionalidad verificada	Estado y evidencia
Resolución de conflictos (Módulo 3, subconjunto MVP)	ConflictSolver.jsx	Colisión línea a línea con marcadores; resolución en flujos merge y rebase (--continue).	Implementado (TUC-UX-05).
Colaboración con remotos (Módulo 2, versión simulada)	GitHubHub.jsx	Push, Pull Request, revisión de código, fusión y pull, sin conexión real con GitHub.	Implementado en versión simulada (TUC-UX-06). La integración OAuth queda para una fase posterior.
Tutoría socrática Nova	AITutor.jsx; shonnyProxyService.js	Pistas basadas en el estado de Git con reglas locales; pasarela opcional a un modelo de lenguaje.	Implementado (TUC-UX-04); por defecto funciona sin servicios externos.
Lecciones animadas	VideoLearning.jsx; LessonPlayer.jsx; lessons/	Seis lecciones con controles, subtítulos, voz opcional y teclado.	Implementado (TUC-UX-02 y TUC-UX-10); contenido validado por pruebas automáticas.
Gamificación y progreso	Dashboard.jsx; learnerStore.js; badges.js	XP, cinco niveles, nueve medallas; progreso guardado en el navegador y reiniciable.	Implementado (TUC-UX-08).
Evaluación formativa	Quizzes.jsx	Cinco preguntas con retroalimentación inmediata.	Implementado.
Accesibilidad (meta WCAG 2.2 AA y DUA)	index.css; App.jsx; Sidebar.jsx	Contraste ≥ 4,5:1, enlace de salto, foco visible, menú móvil con foco atrapado, movimiento reducido.	Auditoría automatizada sin violaciones graves (TUC-UX-11). Falta evaluación manual y con usuarios.

11.1. Resultados de Pruebas Automatizadas y Métricas de Calidad

Las siguientes mediciones se obtuvieron el 17 de octubre de 2026 sobre la versión publicada:

- **Pruebas unitarias (Vitest 5.0.3):** 46 de 46 aprobadas en 412 ms. Cubren el estado del estudiante (persistencia, medallas, niveles), el reproductor de lecciones y la validación de las seis lecciones.
- **Pruebas de extremo a extremo (Playwright):** 12 de 12 casos de uso (TUC-UX-01 a TUC-UX-12) aprobados, incluidos progreso persistente, uso en teléfono de 375 px, reproductor y ausencia de errores de ejecución.

- **Accesibilidad:** auditoría con axe-core 4.13.0 en las seis secciones y en ambos temas sin violaciones de severidad grave o crítica, más pruebas de navegación por teclado. Por tratarse de una auditoría automatizada, se formula como evidencia parcial hacia la meta WCAG 2.2 AA y no como certificación de conformidad.
- **Empaquetado de producción (Vite 8.0.14):** compilación en 445 ms; JavaScript de 102,9 kB y CSS de 6,2 kB comprimidos (gzip).
- **Costo de operación:** alojamiento estático en el plan gratuito de Netlify, sin servidor propio. El costo recurrente para la universidad es nulo mientras se mantenga ese plan.

Pendiente para la etapa de Evaluación: aplicar la escala SUS y la medición de carga cognitiva con estudiantes y docentes de la Escuela de Computación (objetivo específico 5). Como siguiente iteración del MVP se prevé el registro de usuarios y el almacenamiento del progreso en un servidor, lo que exigirá aplicar las consideraciones de protección de datos señaladas en el análisis PESTEL (sección 3).

12. Declaración de Uso de Inteligencia Artificial Generativa (IAG)

En cumplimiento de los lineamientos de la cátedra, el equipo declara el uso de IAG durante la Fase II. No se utilizaron herramientas no-code ni low-code para construir el prototipo.

12.1. Herramientas, Versión y Sitio Oficial

Tabla 15

Herramientas de IAG utilizadas o incluidas en el RED.

Herramienta	Versión	Sitio oficial	Tipo de uso
Claude Code (Anthropic, 2026), agente de programación con el modelo Claude Opus 5.5	Claude Code 2.1.296	https://claude.com/claude-code	Herramienta de desarrollo y documentación usada por el equipo.

Herramienta	Versión	Sitio oficial	Tipo de uso
Pasarela ShonnyProxy para la tutora Nova (opcional)	Interfaz compatible con /v1/chat/completions	Configuración local del proyecto	Funcionalidad incluida en el RED: si se activa, Nova consulta un modelo de lenguaje; por defecto usa reglas locales.

12.2. Procesos y Actividades de la Fase II Donde se Utilizó

Tabla 16

Actividades de la Fase II apoyadas con IAG y supervisión del equipo.

Actividad	Uso de la IAG	Supervisión y verificación del equipo
Diagnóstico del prototipo existente	Revisión del sitio publicado y del código; detección de problemas de diseño adaptable, accesibilidad y videos externos mal enlazados.	El equipo eligió el alcance (lecciones animadas propias y base de UX/accesibilidad).
Planificación	Propuesta de un plan de trece incrementos con criterios de aceptación.	Aprobación del plan y de las decisiones de diseño.
Implementación del prototipo	Escritura de código: progreso persistente, diseño adaptable, accesibilidad, motor y reproductor de lecciones.	Revisión de cada incremento mediante Pull Request antes de integrarlo.
Guiones de las lecciones	Redacción de escenas y subtítulos.	Contraste de cada afirmación con Pro Git (Chacon y Straub, 2014); validación automática de la coherencia de cada escena.
Pruebas y auditorías	Escritura y ejecución de pruebas unitarias, de interfaz y de accesibilidad.	Lectura de resultados; casos de aceptación manuales (TUC).
Documentación e informe	Estructuración del informe de la Fase II, figuras (mapa, bocetos, capturas) y redacción preliminar.	Revisión, corrección y aprobación final del contenido por los autores.

12.3. Beneficios, Ventajas y Precauciones

- **Productividad:** una iteración completa (13 incrementos con pruebas) se realizó en una jornada de trabajo, lo que hizo viable un prototipo vertical dentro del período académico.
- **Calidad verificable:** el uso de IAG se acompañó de pruebas automatizadas que detectaron defectos reales antes de publicar (por ejemplo, un error que dejaba la página en blanco pese a compilar correctamente).

- **Detección de problemas de contenido:** la revisión identificó que los videos externos de la versión anterior estaban mal enlazados (uno inexistente y marcas de tiempo que no correspondían), lo que motivó crear lecciones propias.
- **Precauciones adoptadas:** toda salida de la IAG se trató como borrador sujeto a revisión humana; las referencias bibliográficas y enlaces se verificaron individualmente; los datos personales se excluyeron de los archivos publicados; y la autoría y responsabilidad del recurso son del equipo.

12.4. Prompts Utilizados

Se transcriben textualmente las instrucciones dadas al agente durante la Fase II, en el orden en que se usaron, con su propósito y resultado:

Tabla 17

Prompts utilizados con Claude Code durante la Fase II.

N.º	Prompt (textual)	Propósito y resultado
1	«Tengo este proyecto donde esta deployado en el Netlify: https://git-playground-ucv.netlify.app/, necesito hacer mejoras de este proyecto, ya que es para enseñanza y uso de Git, de manejo de control de versiones, algo que sea visual, amigable, si hace falta hacer vídeos de explicación con Motion-Graphics seria genial, utilicemos todos los /skill que tenemos en conocimiento tomando en cuenta las de shonny-workspace-template donde analizamos el otro dia. Aplicado como Ingeniero de Software, si hace falta documentar ahi tenemos en la carpeta de docs.»	Diagnóstico y plan de mejoras. El agente formuló preguntas de diseño; el equipo eligió: lecciones animadas dentro de la aplicación, prioridad a la base de UX y accesibilidad, y trabajo en una rama separada sin publicar.
2	«Si, adelante»	Autorización para publicar la rama y abrir el Pull Request #1 con los cambios para revisión.
3	«Genial, puedes subir los cambios»	Integración del Pull Request en la rama principal; publicación automática en Netlify y verificación del sitio.
4	«Si, vamos a incluir los laboratorios ya que son referencia para la documentacion del Software»	Incorporación de los laboratorios al repositorio, retirando datos personales y corrigiendo enlaces.

N.º	Prompt (textual)	Propósito y resultado
5	«Ahora tengo el proyecto final que es del: …/docs/PROYECTO, donde tengo las mejoras del desarrollo de la Fase #1, y necesito hacer la Fase #2. Siguiendo las instrucciones que nos comenta la profesora, ya que este proyecto es académico y educativo como podrás ver del directorio. […]»	Elaboración de este informe de la Fase II a partir de las instrucciones de la cátedra y del informe corregido de la Fase I.

Referencias

Álvarez González, L., Carballo Barrera, Y., Collazos Ordoñez, C., Echenagusía, J., Gutiérrez Hernández, R., Hernández Bieliukas, Y., Hernández Saldivar, F., Muñoz Arteaga, J., Solano Alegría, A., y Velázquez Amador, C. (2014). *Objetos de aprendizaje de contenidos abiertos accesibles: Del diseño a la reutilización*. Iniciativa Latinoamericana de Libros de Texto Abiertos (LATIn). https://www.researchgate.net/publication/312039527_Objetos_de_Aprendizaje_de_Contenidos_Abiertos_Accesibles_del_Diseno_a_la_Reutilizacion

Anderson, L. W., y Krathwohl, D. R. (Eds.). (2001). *A taxonomy for learning, teaching, and assessing: A revision of Bloom's taxonomy of educational objectives*. Longman.

Anthropic. (2026). *Claude Code* (Versión 2.1.296) [Software]. https://claude.com/claude-code

Brown, T. (2008). Design thinking. *Harvard Business Review, 86*(6), 84–92. https://emprendedoresupa.wordpress.com/wp-content/uploads/2010/08/p02_brown-design-thinking.pdf

CAST. (2018). *Universal Design for Learning guidelines version 2.2*. https://udlguidelines.cast.org

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Costa Montenegro, E., Díaz Otero, F., Caeiro Rodríguez, M., Cuiñas Gómez, I., Mariño Espiñeira, P., y Fernández Iglesias, M. (2016). Evaluación de la implantación de la metodología Design Thinking en una asignatura de proyectos. En *Libro de Actas IN-RED 2016: II Congreso Nacional de Innovación Educativa y de Docencia en Red*. Universitat Politècnica de València. https://doi.org/10.4995/INRED2016.2016.4357

Cottle, P. (s.f.). *Learn Git Branching* [Aplicación web]. Recuperado el 5 de octubre de 2026, de https://learngitbranching.js.org

Deque Systems. (2026). *axe-core* (Versión 4.13.0) [Software]. https://github.com/dequelabs/axe-core

Deterding, S., Dixon, D., Khaled, R., y Nacke, L. (2011). From game design elements to gamefulness: Defining "gamification". En *Proceedings of the 15th International Academic MindTrek Conference* (pp. 9–15). Association for Computing Machinery. https://doi.org/10.1145/2181037.2181040

Díaz-Barriga Arceo, F., y Hernández Rojas, G. (2002). *Estrategias docentes para un aprendizaje significativo: Una interpretación constructivista* (2.ª ed.). McGraw-Hill.

Git Project. (s.f.). *Git documentation*. Recuperado el 2 de octubre de 2026, de https://git-scm.com/doc

Gros Salvat, B. (1997). *Diseños y programas educativos: Pautas pedagógicas para la elaboración de software*. Ariel.

Haaranen, L., y Lehtinen, T. (2015). Teaching Git on the side: Version control system as a course platform. En *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education* (pp. 87–92). Association for Computing Machinery. https://doi.org/10.1145/2729094.2742608

Hattie, J., y Timperley, H. (2007). The power of feedback. *Review of Educational Research, 77*(1), 81–112. https://doi.org/10.3102/003465430298487

Herrera Batista, M. Á. (2006). Consideraciones para el diseño didáctico de ambientes virtuales de aprendizaje: Una propuesta basada en las funciones cognitivas del aprendizaje. *Revista Iberoamericana de Educación, 38*(5), 1–20. https://doi.org/10.35362/rie3852623

IBM. (s.f.). *Low-code vs. no-code: What's the difference?* Recuperado el 17 de octubre de 2026, de https://www.ibm.com/think/topics/low-code-vs-no-code

Kolb, D. A. (1984). *Experiential learning: Experience as the source of learning and development*. Prentice-Hall.

Lodi, M., y Martini, S. (2021). Computational thinking, between Papert and Wing. *Science & Education, 30*(4), 883–908. https://doi.org/10.1007/s11191-021-00202-5

Mayer, R. E. (2021). *Multimedia learning* (3.ª ed.). Cambridge University Press. https://doi.org/10.1017/9781316941355

Meta Platforms. (2026). *React* (Versión 19.2.6) [Software]. https://react.dev

Microsoft. (2026). *Playwright* (Versión 1.57.0) [Software]. https://playwright.dev

Mozilla Developer Network. (s.f.). *Web Speech API*. Recuperado el 17 de octubre de 2026, de https://developer.mozilla.org/es/docs/Web/API/Web_Speech_API

Netlify. (s.f.). *Netlify* [Plataforma de alojamiento web]. Recuperado el 17 de octubre de 2026, de https://www.netlify.com

Novak, J. D., y Gowin, D. B. (1988). *Aprendiendo a aprender*. Martínez Roca.

Ospina, M., y Carballo Barrera, Y. (2022). Revisión de técnicas y tecnologías para la construcción del producto mínimo viable para emprendimientos universitarios. En *Novena Conferencia Nacional de Computación, Informática y Sistemas (CoNCISa 2022)* (pp. 6–10). Universidad Central de Venezuela. https://www.researchgate.net/publication/377235110_Revision_de_Tecnicas_y_Tecnologias_para_la_Construccion_del_Producto_Minimo_Viable_en_el_Contexto_del_Emprendimiento_Universitario

República Bolivariana de Venezuela. (2001, 28 de febrero). *Decreto con Fuerza de Ley sobre Mensajes de Datos y Firmas Electrónicas* (Decreto N.º 1.204). Gaceta Oficial N.º 37.148. https://conatel.gob.ve/wp-content/uploads/2024/08/PDF-Ley-sobre-Mensajes-de-Datos-y-Firmas-Electronicas.pdf

República Bolivariana de Venezuela. (2013, 17 de octubre). *Ley de Infogobierno*. Gaceta Oficial N.º 40.274.

Shneiderman, B., y Plaisant, C. (2005). *Designing the user interface: Strategies for effective human-computer interaction* (4.ª ed.). Pearson Addison-Wesley.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Spinellis, D. (2012). Git. *IEEE Software, 29*(3), 100–101. https://doi.org/10.1109/MS.2012.61

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science, 12*(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4

Torres, J., Riera, R., y Darder, J. (2026). *GitPlayground: Aprende Git interactivo* [Recurso educativo digital]. https://git-playground-ucv.netlify.app

UNESCO. (2019). *Recomendación sobre los recursos educativos abiertos (REA)*. https://unesdoc.unesco.org/ark:/48223/pf0000373755

VoidZero. (2026a). *Vite* (Versión 8.0.14) [Software]. https://vite.dev

VoidZero. (2026b). *Vitest* (Versión 5.0.3) [Software]. https://vitest.dev

Vygotsky, L. S. (1978). *Mind in society: The development of higher psychological processes*. Harvard University Press.

Wieluch, S., y Morr, S. (s.f.). *Oh My Git!* [Videojuego de código abierto]. Recuperado el 5 de octubre de 2026, de https://ohmygit.org

Wood, D., Bruner, J. S., y Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry, 17*(2), 89–100. https://doi.org/10.1111/j.1469-7610.1976.tb00381.x

Woolf, B. P. (2009). *Building intelligent interactive tutors: Student-centered strategies for revolutionizing e-learning*. Morgan Kaufmann.

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2* [Recomendación W3C]. https://www.w3.org/TR/WCAG22/

Zagalsky, A., Feliciano, J., Storey, M.-A., Zhao, Y., y Wang, W. (2015). The emergence of GitHub as a collaborative platform for education. En *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing* (pp. 1906–1917). Association for Computing Machinery. https://doi.org/10.1145/2675133.2675284